

UNIVERSIDAD NACIONAL DE COLOMBIA
INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Tarea 15

Diseño de un computador von Neumann sobre una MANET: emulador de una
MANET heterogénea

Presentado por:

Rodríguez Rodríguez Daniel Santiago
Olarte Ramírez Maicol Sebastián

Lenguajes de Programación

Presentado a: Jorge Eduardo Ortiz Triviño

Contenido de la entrega

Documento único con todos los entregables exigidos a una tarea obligatoria automatizada (Lineamientos operativos del curso, sección 3.1.1). Cada literal corresponde a un capítulo:

- (a) Marco teórico
- (b) Descripción y justificación del problema
- (c) Diseño de la solución
- (d) Código fuente completo
- (e) Manual de usuario y manual técnico
- (f) Experimentación y análisis de resultados

(A) MARCO TEÓRICO

1. La arquitectura de von Neumann

En el *First Draft of a Report on the EDVAC* (1945), John von Neumann describió un computador formado por una unidad aritmética, una unidad de control, una memoria que guarda **tanto los datos como el programa**, y dispositivos de entrada y salida, comunicados por buses. La idea del programa almacenado permite que el mismo hardware ejecute cualquier algoritmo cargando otra secuencia de instrucciones en memoria. La unidad de control repite indefinidamente el ciclo **buscar** → **decodificar** → **ejecutar**: lee la instrucción apuntada por el contador de programa (PC), la interpreta y genera las señales para la ALU, los registros o la memoria (Peña, 2006, sec. 2.5).

Dentro de ese esquema común caben diseños muy distintos. Los elementos que la Tarea 15 exige variar entre “fabricantes” son:

Dimensión	Alternativas
Modelo de operandos	Máquina de acumulador (un registro implícito, p. ej. EDSAC, PDP-8), de pila o cero direcciones (Burroughs B5000, JVM, chips Forth), de registros de propósito general (IBM 360, RISC).
Tamaño de palabra	8, 16, 32, 64 bits: determina el rango de los enteros y el ancho de registros y buses.
Direccionamiento de memoria	Por byte o por palabra; ancho del bus de direcciones (capacidad máxima).
Orden de los bytes (<i>endianness</i>)	Big-endian (el byte más significativo en la dirección menor: IBM 360, Motorola 68k) o little-endian (el menos significativo primero: x86).
Formato de instrucción	Longitud fija (RISC) o variable (CISC); número de campos y modos de direccionamiento.
Banderas	Conjunto de indicadores de estado (cero, negativo, acarreo, desbordamiento...) o ninguno (las máquinas de pila suelen dejar el resultado de la comparación en la pila).
Reloj	Instrucciones por unidad de tiempo.

Cada combinación define un **conjunto de instrucciones (ISA)** distinto; un programa binario de una máquina no tiene significado en otra, y su lenguaje ensamblador (la representación simbólica uno a uno de ese binario) también es distinto. Por eso, en una red de computadores heterogéneos, lo que se comparte no son programas sino **mensajes con un formato acordado**.

2. Heterogeneidad y representación externa de datos

Cuando dos máquinas con distinto tamaño de palabra o distinto orden de bytes intercambian un entero, deben acordar una representación externa. En Internet se usa el **orden de red** (big-endian), definido para los protocolos TCP/IP, y funciones como `htonl/ntohl` convierten entre el orden del anfitrión y el de red. Estándares como XDR (RFC 4506) o ASN.1 generalizan esta idea. El proceso de convertir datos internos a esa representación se llama **serialización** (*marshalling*). En esta tarea:

- una máquina little-endian debe *invertir el orden de los bytes* de sus enteros (instrucción BSWAP del ORINOCO-32);
- una máquina de 16 bits debe representar enteros de 32 bits con *dos palabras* y aritmética de precisión múltiple (propagando el acarreo, como en el CARIBE-16);
- una máquina direccionable por palabra debe decidir cómo mapear bytes a palabras (el CARIBE-16 usa un byte por palabra en su interfaz de red de 8 bits).

3. Redes móviles ad hoc (MANET)

Una MANET (*Mobile Ad hoc NETwork*) es una red formada por nodos móviles que se comunican por radio **sin infraestructura fija** (sin enrutadores ni puntos de acceso). Cada nodo es a la vez anfitrión y enrutador: si el destino está fuera del alcance de radio, los nodos intermedios reenvían los paquetes (**comunicación multi-salto**). El grupo de trabajo MANET del IETF (RFC 2501, 1999) resume sus características:

- *Topología dinámica*: los nodos se mueven y los enlaces aparecen y desaparecen de forma impredecible.
- *Enlaces de ancho de banda limitado y con pérdidas*, variables y posiblemente asimétricos.
- *Energía limitada*: los nodos funcionan con baterías; cada transmisión cuesta energía.
- *Seguridad física limitada* y operación distribuida sin administración central.

Aplicaciones típicas: comunicaciones militares y de emergencia, redes vehiculares (VANET), redes de sensores y redes comunitarias. El alcance de radio define un **grafo de conectividad** (modelo de disco unitario): hay enlace entre dos nodos si su distancia es menor que el alcance. Si el grafo no es conexo, la red está **particionada** y algunos destinos son inalcanzables.

4. Enrutamiento en MANET

Familia	Idea	Ejemplos	Ventaja / desventaja
Proactivos (dirigidos por tabla)	Cada nodo mantiene rutas a todos los destinos intercambiando periódicamente su tabla.	DSDV, OLSR (RFC 3626)	Ruta inmediata / tráfico de control constante aunque no haya datos.
Reactivos (bajo demanda)	La ruta se busca solo cuando se necesita, inundando una solicitud.	AODV (RFC 3561), DSR (RFC 4728)	Poco control en reposo / latencia del primer paquete.
Híbridos	Proactivos dentro de una zona, reactivos fuera.	ZRP	Compromiso entre ambos.

4.1 AODV (Ad hoc On-Demand Distance Vector)

Propuesto por Perkins y Royer (1999) y estandarizado en el RFC 3561, AODV funciona así:

1. *Descubrimiento*: el origen que no tiene ruta difunde un **RREQ** (*route request*) con un identificador. Cada nodo que lo recibe por primera vez registra una **ruta inversa** hacia el origen (a través del vecino que se lo entregó) y lo retransmite; las copias repetidas se descartan.
2. *Respuesta*: el destino (o un nodo con ruta fresca) contesta con un **RREP** unicast que viaja por la ruta inversa; cada nodo del camino instala la **ruta directa** hacia el destino.
3. *Mantenimiento*: las rutas tienen tiempo de vida y se renuevan con el uso. Si un enlace se rompe, el nodo que lo detecta invalida las rutas afectadas y envía un RERR al origen o intenta una **reparación local**.
4. Cada nodo solo conoce el *siguiente salto* (vector de distancias), y los números de secuencia evitan bucles.

5. Capa de enlace inalámbrica

En IEEE 802.11 cada trama unicast debe ser confirmada con un ACK; si no llega, se retransmite hasta un número máximo de intentos (7 por omisión para tramas cortas). Si todos fallan, la capa de enlace informa una **ruptura de enlace** a la capa de red, aunque el vecino siga dentro del alcance (“ruptura falsa” por pérdidas). Los broadcast (como los RREQ) no se confirman. Si cada intento se pierde independientemente con probabilidad p , un salto falla con probabilidad p^k para k intentos, y un camino de h saltos entrega el paquete con probabilidad $(1 - p^k)^h$.

6. Modelos de movilidad

El modelo más usado para evaluar protocolos MANET es el **random waypoint** (Johnson y Maltz, 1996): cada nodo elige un destino aleatorio uniforme en el área, se desplaza hacia él a velocidad constante, espera un tiempo de pausa y repite. Camp, Boleng y Davies (2002) muestran que sus propiedades no son triviales: la densidad de nodos se concentra en el centro del área, y la velocidad cambia la frecuencia de rupturas de enlaces. Por eso, en la evaluación, **la movilidad puede mejorar la conectividad media** de una red dispersa (reúne nodos que estaban aislados) y al mismo tiempo aumentar la tasa de rupturas.

7. Métricas de desempeño

Métrica	Definición
PDR (<i>Packet Delivery Ratio</i>)	paquetes de datos entregados / paquetes de datos enviados por las aplicaciones.
Latencia de extremo a extremo	tiempo desde que la aplicación envía hasta que el destino recibe (incluye descubrimiento de ruta y retransmisiones).
Sobrecarga de enrutamiento	transmisiones de control (RREQ + RREP) por transmisión de datos.
Energía	costo acumulado de transmitir y recibir por nodo.

8. Emulación y simulación de eventos discretos

Un **simulador** de redes (ns-3, OMNeT++) modela el comportamiento de los protocolos; un **emulador** ejecuta el software real sobre un modelo del entorno. Esta tarea combina ambos: las CPU de los nodos se **emulan** instrucción por instrucción (ejecutan programas binarios reales de cada ISA), mientras el medio de radio y el enrutamiento se **simulan** con un modelo de tiempo discreto (ticks). Todas las fuentes de aleatoriedad (posiciones, movilidad, pérdidas) se derivan de una semilla, de modo que cada experimento es reproducible.

9. Referencias

1. von Neumann, J. (1945). *First Draft of a Report on the EDVAC*. Moore School of Electrical Engineering, University of Pennsylvania.
2. Peña, R. (2006). *De Euclides a Java*. Nivola. Sec. 2.5, “Cómo funciona un computador”.
3. Corson, S. y Macker, J. (1999). *Mobile Ad hoc Networking (MANET): Routing Protocol Performance Issues and Evaluation Considerations*. RFC 2501, IETF.
4. Perkins, C., Belding-Royer, E. y Das, S. (2003). *Ad hoc On-Demand Distance Vector (AODV) Routing*. RFC 3561, IETF.

5. Perkins, C. y Royer, E. (1999). Ad-hoc on-demand distance vector routing. *Proc. IEEE WMCSA*, 90–100.
6. Johnson, D. y Maltz, D. (1996). Dynamic source routing in ad hoc wireless networks. En *Mobile Computing*, Kluwer, 153–181.
7. Camp, T., Boleng, J. y Davies, V. (2002). A survey of mobility models for ad hoc network research. *Wireless Communications and Mobile Computing*, 2(5), 483–502.
8. Clausen, T. y Jacquet, P. (2003). *Optimized Link State Routing Protocol (OLSR)*. RFC 3626, IETF.
9. Cohen, D. (1981). On holy wars and a plea for peace. *IEEE Computer*, 14(10), 48–54. (Origen de los términos big/little-endian en computación.)
10. Eisler, M. (2006). *XDR: External Data Representation Standard*. RFC 4506, IETF.
11. Tanenbaum, A. S. y Austin, T. (2013). *Structured Computer Organization* (6.^a ed.). Pearson. Caps. 4–5 (ISA, formatos de instrucción, máquinas de pila).
12. IEEE Std 802.11-2020. *Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*.

(B) DESCRIPCIÓN Y JUSTIFICACIÓN DEL PROBLEMA

1. Enunciado

1. *Diseño de un emulador de una MANET heterogénea* compuesta por (al menos) tres nodos (computadores) con arquitectura John von Neumann de diferentes “fabricantes”, lo cual implica que los registros, la memoria, las banderas, los buses, la CPU, etc. no son iguales y que su conjunto de microinstrucciones en binario y su lenguaje ensamblador son distintos.
2. (Opcional, dificultad 2) Emulador de un computador virtual von Neumann que se ejecute sobre la MANET, con al menos 40 microinstrucciones virtuales.
3. (Opcional, continuación) Ensamblador y Enlazador-Cargador de la máquina virtual.

Alcance de esta entrega: se resuelve el punto 1 (obligatorio) completo — diseño *e* implementación del emulador de la MANET heterogénea. Los puntos 2 y 3 son opcionales y no se abordan.

2. Descripción del problema

Se necesita un programa que reproduzca, en un solo computador anfitrión, el funcionamiento de una red móvil ad hoc cuyos nodos son computadores von Neumann **de fabricantes distintos**. El problema tiene dos mitades que deben encajar:

2.1 Los nodos: tres computadores realmente distintos

No basta con dar nombres diferentes a una misma máquina: cada fabricante debe tener su propia organización interna y su propio lenguaje. Se diseñaron tres:

	ANDES-64	ORINOCO-32	CARIBE-16
Fabricante (ficticio)	Industrias Andinas de Cómputo	Orinoco Systems S.A.	Compañía Caribeña de Calculadoras
Modelo de operandos	16 registros de propósito general	pila (cero direcciones)	acumulador + registro índice
Palabra	64 bits	32 bits	16 bits
Memoria	4 KB por byte, big-endian, caché L1/L2	8 KB por byte, little-endian	1024 palabras, direccionable por palabra
Bus de direcciones	12 bits	13 bits	10 bits
Instrucciones	71, longitud variable 16–40 bits	39, longitud variable 1–5 bytes	32, longitud fija 16 bits
Banderas	Z, N, C, V, I	O (desbordamiento), E (error de pila)	Z, N, C
Subrutinas	CALL/RET con pila en RAM	pila de retorno interna (16)	JSR guarda el retorno en la subrutina (PDP-8)
Multiplicación	sí (64 bits)	sí (32 bits)	no
Ensamblador	MOVI R1, 5	push buf pushb 1 add loadb	LDA #5 · ADD var
Reloj	8 instr./tick	5 instr./tick	3 instr./tick

El ANDES-64 es el computador diseñado por el grupo en la Tarea 9 (e implementado para las Tareas 10 y 17), reutilizado aquí como uno de los tres fabricantes.

2.2 La red: una MANET emulada

- Los nodos tienen posición en un plano, alcance de radio y, opcionalmente, movimiento.
- Los mensajes viajan por radio con retardo y pérdidas; si el destino no está al alcance, otros nodos —de *cualquier* fabricante— los reenvían.
- Las rutas se descubren y se reparan dinámicamente (protocolo tipo AODV).
- Los programas de cada CPU, escritos en su propio ensamblador, usan su tarjeta de red para enviar y recibir; la heterogeneidad obliga a definir un *formato de mensaje común* y a que cada programa lo traduzca a su representación interna (tamaño de palabra y orden de bytes).

3. Justificación

- *Relación con el curso.* La Tarea 9 pidió que el computador tuviera “dispositivos de comunicaciones que le permitan unirse a redes de computadores convencionales”. Esta tarea lleva esa idea al caso extremo: una red sin infraestructura en la que los computadores ni siquiera comparten el lenguaje de máquina. Obliga a pensar qué significa “lenguaje” en cada nivel: el de máquina (binario), el ensamblador (simbólico) y el protocolo (el lenguaje común de la red).
- *Heterogeneidad real.* Internet funciona precisamente entre máquinas heterogéneas (x86 little-endian, antiguos PowerPC big-endian, microcontroladores de 8/16 bits en IoT). El problema de representación de datos que aquí aparece —orden de bytes, tamaño de palabra— es el mismo que resolvieron XDR, ASN.1 o el “orden de red” de TCP/IP.
- *Vigencia de las MANET.* Redes de emergencia, de vehículos, de drones y de sensores dependen de enrutamiento multi-salto sin infraestructura; entender el efecto de la movilidad y de las pérdidas en la entrega es un conocimiento básico de la ingeniería de sistemas.
- *Emulación como herramienta.* Construir la red en software permite experimentar con escenarios reproducibles (misma semilla, mismos resultados) que serían costosos con hardware real.

4. Objetivos

General

Diseñar e implementar un emulador de una MANET heterogénea de al menos tres computadores von Neumann de fabricantes distintos, que ejecuten programas binarios de sus propias ISA y cooperen a través de la red.

Específicos

1. Diseñar tres arquitecturas von Neumann con registros, memoria, banderas, buses, CPU, formatos de instrucción, código binario y lenguaje ensamblador diferentes, e implementar su emulador y su ensamblador.
2. Diseñar una interfaz de red común (NIC + PDU canónica) que oculte las diferencias de las máquinas al medio pero no a los programas.
3. Implementar el medio de radio (alcance, retardo, pérdidas, reintentos), la movilidad y un protocolo de enrutamiento reactivo tipo AODV.

4. Escribir en los tres lenguajes ensambladores programas que cooperen (servidores, cliente, coordinador de un cálculo distribuido).
5. Medir el desempeño (PDR, latencia, sobrecarga, energía, tiempo de servicio) en al menos tres escenarios y analizar el efecto de la movilidad, las pérdidas y la heterogeneidad.

5. Restricciones

- Implementación en Python 3 con *solo la biblioteca estándar* (lineamiento 3.3.1); la visualización usa tkinter, incluido en la instalación estándar.
- Los nodos se emulan a nivel de instrucción; la red es un modelo de tiempo discreto (1 tick = 10 ms simulados).
- El protocolo de enrutamiento es una versión simplificada de AODV (sin números de secuencia de destino ni RERR; ver diseño).

(C) DISEÑO DE LA SOLUCIÓN

1. Arquitectura del emulador

El emulador se organiza en capas, siguiendo el modelo de una pila de protocolos. Las tres CPU solo se parecen en su frontera con la red: todas tienen una tarjeta de red (NIC) que transmite **bytes**; por encima de esa frontera, cada máquina es completamente distinta.

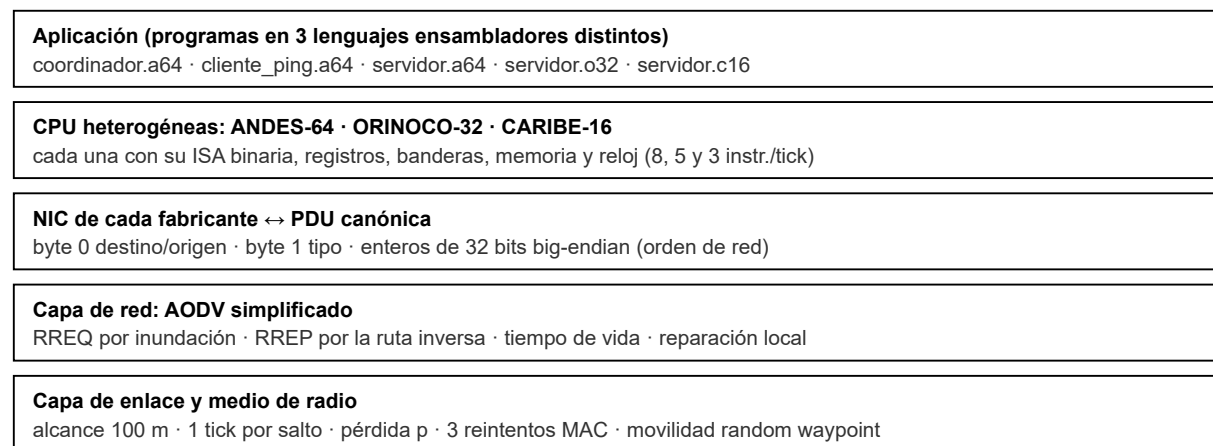


Figura 1. Capas del emulador de la MANET.

2. Diseño de los tres computadores

2.0 Comparación de los componentes von Neumann de cada fabricante

Los tres nodos respetan la arquitectura de John von Neumann (CPU con unidad de control y ALU, memoria única para programa y datos, unidad de entrada/salida y buses), pero ninguno de sus componentes coincide con el de los otros dos:

Componente	ANDES-64	ORINOCO-32	CARIBE-16
CPU (modelo de operandos)	registros de propósito general (3 operandos)	pila (0 operandos)	acumulador (1 operando)
Registros visibles	R0–R15 de 64 bits	ninguno (pila de datos de 32 entradas)	ACC y X de 16 bits
Registros de control	PC 12, IR 64, MAR 12, MDR 64, FR 8, SP	PC 13, IR ≤ 40 , SP 5, RSP 4	PC 10, IR 16
Banderas	Z, N, C, V, I	O, E	Z, N, C
Memoria	4 KB por byte, big-endian, caché L1/L2	8 KB por byte, little-endian	1 024 palabras de 16 bits
Bus de datos	64 bits	32 bits	16 bits
Bus de direcciones	12 bits (4 096 bytes)	13 bits (8 192 bytes)	10 bits (1 024 palabras)
Bus de control	12 señales: lectura, escritura, habilitación de memoria, de caché L1 y L2, E/S, NIC, interrupción, reconocimiento de interrupción, reloj, reinicio, espera	8 señales: lectura, escritura, byte/palabra, E/S, NIC, reloj, reinicio, error	6 señales: lectura, escritura, E/S, NIC, reloj, reinicio
Instrucciones	71, de 16 a 40 bits (6 formatos)	39, de 1 a 5 bytes (4 formatos)	32, fijas de 16 bits (1 formato)
Subrutinas	CALL/RET con pila en RAM	pila de retorno interna	JSR/JMI (retorno guardado en la subrutina)
Interfaz de red	NSEND/NRECV sobre bytes	send/recv sobre bytes + bswap	NSND/NRCV, un byte por palabra
Reloj	8 instrucciones/tick	5 instrucciones/tick	3 instrucciones/tick

ANDES-64 — Industrias Andinas de Cómputo

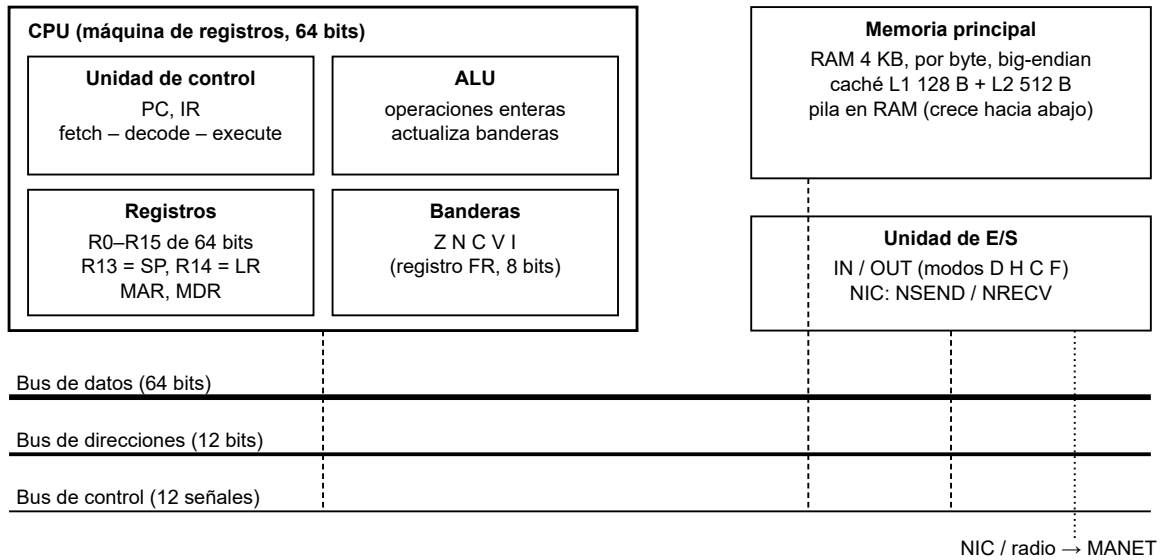


Figura 2a. Diagrama de bloques del ANDES-64.

ORINOCO-32 — Orinoco Systems S.A.

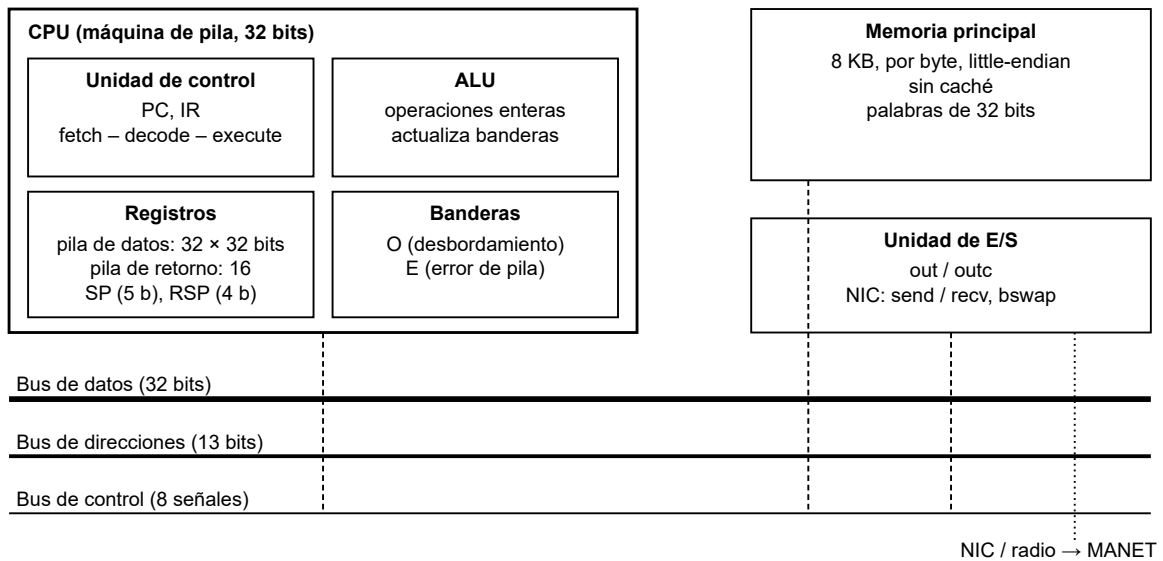


Figura 2b. Diagrama de bloques del ORINOCO-32.

CARIBE-16 — Compañía Caribeña de Calculadoras

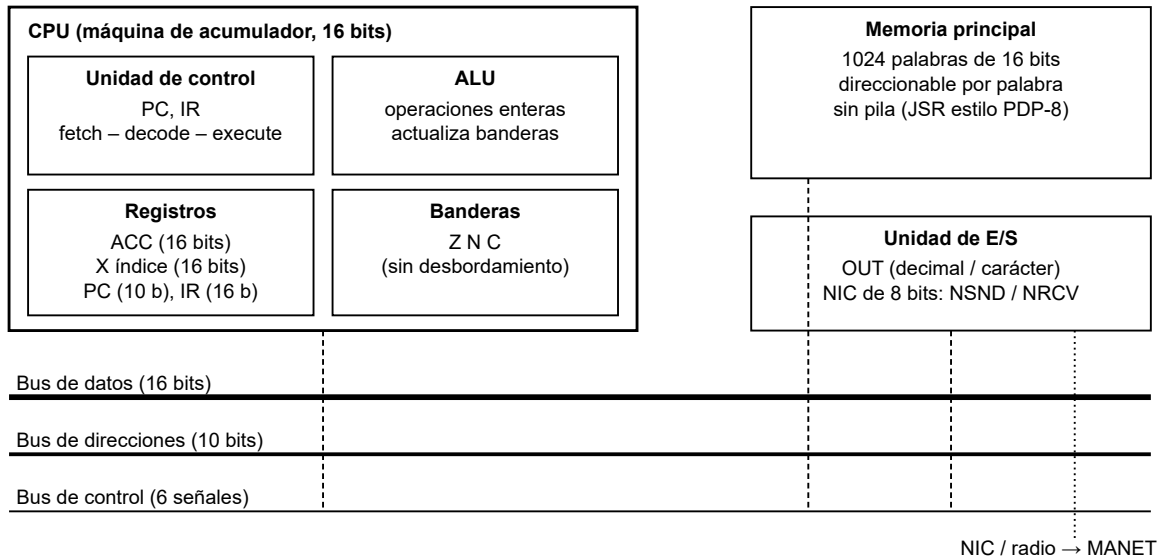


Figura 2c. Diagrama de bloques del CARIBE-16.

2.1 ANDES-64 (Industrias Andinas de Cómputo)

Es el computador de la Tarea 9, implementado en las Tareas 10/17 (codigo/andes64/): máquina de registros de 64 bits, 16 registros (R13 = SP, R14 = LR), banderas Z N C V I, RAM de 4 KB direccionable por byte en orden big-endian, cachés L1 (128 B) y L2 (512 B), bus de direcciones de 12 bits y 71 instrucciones en 6 formatos de 16 a 40 bits. Su NIC se maneja con:

```
NSEND Rn, buf      ; transmite Rn bytes desde buf      (formato F5, 36 bits)
NRECV Rn, buf      ; copia un paquete en buf; Rn ← su longitud (0 si no hay)
```

2.2 ORINOCO-32 (Orinoco Systems S.A.)

Componente	Diseño
CPU	Máquina de pila: la ALU toma sus operandos de la cima de una pila de datos de 32 entradas implementada en registros internos, y deja allí el resultado. Direcciones de retorno en una pila separada de 16 entradas.
Registros	PC (13 bits), SP de datos (5 bits), RSP de retorno (4 bits), IR (hasta 40 bits). No hay registros de propósito general visibles.
Banderas	O (desbordamiento aritmético de 32 bits) y E (error de pila). Las comparaciones (EQ, LT, GT) dejan 1/0 en la pila y los saltos condicionales consumen la cima.
Memoria y buses	8 KB direccionables por byte, palabras de 32 bits en little-endian ; bus de direcciones de 13 bits, bus de datos de 32 bits.
Instrucciones	39, opcode de 8 bits; longitudes 1, 2, 3 o 5 bytes.
NIC	<code>send (dir n --) y recv (dir -- n);</code> bswap invierte los 4 bytes de la cima para convertir al orden de red.

Nemónico	Opcode	Bytes	Nemónico	Opcode	Bytes
nop	0x00	1	shr	0x2D	1
halt	0x01	1	eq	0x30	1
push	0x10	5	lt	0x31	1
pushb	0x11	2	gt	0x32	1
dup	0x12	1	jmp	0x40	3
drop	0x13	1	jz	0x41	3
swap	0x14	1	jnz	0x42	3
over	0x15	1	call	0x43	3
rot	0x16	1	ret	0x44	1
add	0x20	1	load	0x50	1
sub	0x21	1	store	0x51	1
mul	0x22	1	loadb	0x52	1
div	0x23	1	storeb	0x53	1
mod	0x24	1	in	0x60	1
neg	0x25	1	out	0x61	1
and	0x28	1	outc	0x62	1
or	0x29	1	send	0x70	1
xor	0x2A	1	recv	0x71	1
not	0x2B	1	bswap	0x72	1
shl	0x2C	1			

2.3 CARIBE-16 (Compañía Caribeña de Calculadoras)

Componente	Diseño
CPU	Máquina de acumulador: toda operación aritmética o lógica es $ACC \leftarrow ACC \text{ op operando}$. Sin multiplicación ni pila.
Registros	ACC (16), X índice (16), PC (10), IR (16).
Banderas	Z, N y C (acarreo en ADD/ADC; préstamo en SUB/CMP).
Memoria y buses	1024 palabras de 16 bits, direccionable por palabra ; bus de direcciones de 10 bits.
Instrucciones	32, longitud fija de 16 bits: opcode 5 · M 1 (0 directo, 1 inmediato) · operando 10. Direccionamiento indexado con opcodes propios (LDAX/STAX).
Subrutinas	JSR <i>s</i> guarda el PC de retorno en la palabra <i>s</i> y salta a <i>s</i> +1; JMI <i>s</i> retorna (salto indirecto), como en el PDP-8.
NIC	Interfaz de 8 bits: NSND <i>buf</i> transmite ACC palabras desde <i>buf</i> enviando el byte bajo de cada una; NRCV <i>buf</i> deja un byte por palabra y $ACC \leftarrow \text{longitud}$.

Nemónico	Opcode	Inmediato	Nemónico	Opcode	Inmediato
HLT	0		JZ	16	
LDA	1	#	JNZ	17	
STA	2		JN	18	
ADD	3	#	JC	19	
SUB	4	#	JSR	20	
AND	5	#	JMI	21	
OR	6	#	CMP	22	#
XOR	7	#	IN	23	
SHL	8	#	OUT	24	#
SHR	9	#	NSND	25	
LDX	10	#	NRCV	26	
STX	11		NOP	27	
ADX	12	#	ADC	28	#
LDAX	13		CPX	29	#
STAX	14		TAX	30	
JMP	15		TXA	31	

2.4 Formatos de instrucción comparados

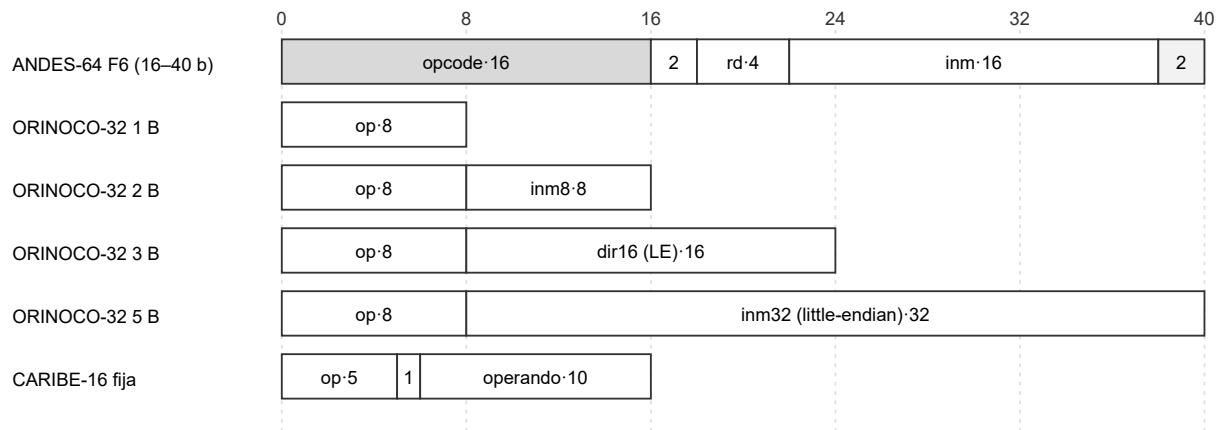


Figura 2. Formatos de instrucción de los tres fabricantes (en bits).

El mismo cálculo ($1 + 2 + \dots + n$) se escribe de forma muy distinta en cada lenguaje ensamblador:

<pre> ; ANDES-64 16 (32 bits en 2 palabras) suma: CMP R4, R2 LDA n JG fin_suma SUB i ADD R3, R3, R4 fin INC R4 LDA lo JMP suma ADD i STA lo LDA hi ADC #0 ; acarreo STA hi LDA i ADD #1 STA i JMP bucle </pre>	<pre> ; ORINOCO-32 (pila) pushb 0 swap (s n) bucle: dup jz fin_suma dup rot add swap (s+n n) pushb 1 sub (s+n n-1) jmp bucle </pre>	<pre> ; CARIBE- bucle: JC </pre>
---	---	----------------------------------

3. Interfaz común: NIC y PDU canónica

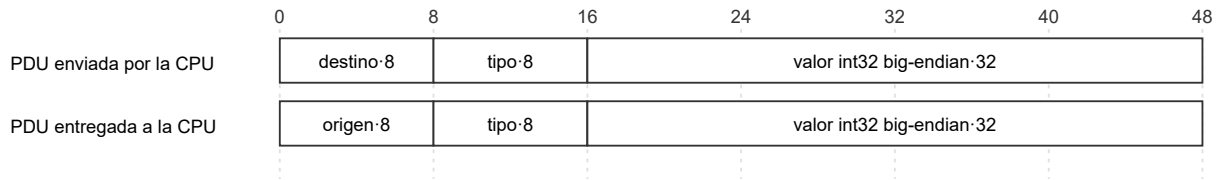


Figura 3. Unidad de datos de protocolo (PDU) que intercambian las aplicaciones.

Tipo	Nombre	Semántica
1	SUMA(n)	Solicita al destino calcular $1 + 2 + \dots + n$.
2	RESULTADO(v)	Respuesta a SUMA.
3	PING(k)	Solicitud de eco con número de secuencia k.
4	PONG(k)	Respuesta de eco.

Al transmitir, el byte 0 es el **destino**; la NIC lo retira y la red entrega al receptor el mismo mensaje con el byte 0 reemplazado por el **origen**. Así, un servidor responde simplemente cambiando el tipo y reenviando el búfer. Los enteros van en 32 bits big-endian (“orden de red”). Traducción en cada máquina:

- *ANDES-64* (big-endian, 64 bits): arma el entero byte a byte con desplazamientos (`LEER_VALOR`, `ESCRIBIR_VALOR`) y extiende el signo de 32 a 64 bits.
- *ORINOCO-32* (little-endian): `load` de 4 bytes seguido de `bswap`; al escribir, `bswap` y `store`.
- *CARIBE-16* (16 bits, por palabra): cada byte ocupa una palabra; el valor de 32 bits se reconstruye como dos palabras (hi:lo) con `SHL #8` y `OR`.

4. Diseño de la red

4.1 Medio de radio y capa de enlace

- Enlace entre i y j si $\text{distancia}(i, j) \leq \min(\text{alcance}_i, \text{alcance}_j)$ (100 m).
- Cada transmisión de un salto llega en el tick siguiente; se pierde con probabilidad p (independiente).
- Unicast: hasta 3 intentos (ACK implícito); si fallan todos o el vecino está fuera de alcance, se notifica ruptura. Broadcast: un solo intento, cada vecino lo recibe o no independientemente.
- Energía: 1 unidad por transmisión (incluidas retransmisiones), 0,5 por recepción.

4.2 Enrutamiento (AODV simplificado)

Parámetro	Valor	Significado
TTL	12	Salto s mximos de un RREQ.
VIDA_RUTA	300 ticks	Tiempo de vida de una ruta sin uso (3 s).
ESPERA_RREP	40 ticks	Espera antes de reintentar el descubrimiento.
MAX_RREQ	3	Intentos de descubrimiento antes de descartar los paquetes pendientes.

Tabla de rutas por nodo: destino $\rightarrow$ [siguiente salto, saltos, expira]. Se aprende la ruta inversa con cada RREP, la directa con cada RREP y tambin la ruta de vuelta con cada paquete de datos recibido. Simplificaciones respecto del RFC 3561: no se usan nmeros de secuencia de destino (las rutas se reemplazan por una de menos saltos o del mismo vecino) y, en vez de RERR, el nodo que detecta la ruptura hace **reparacin local**: borra las rutas que usaban ese vecino, guarda el paquete y busca una nueva ruta.

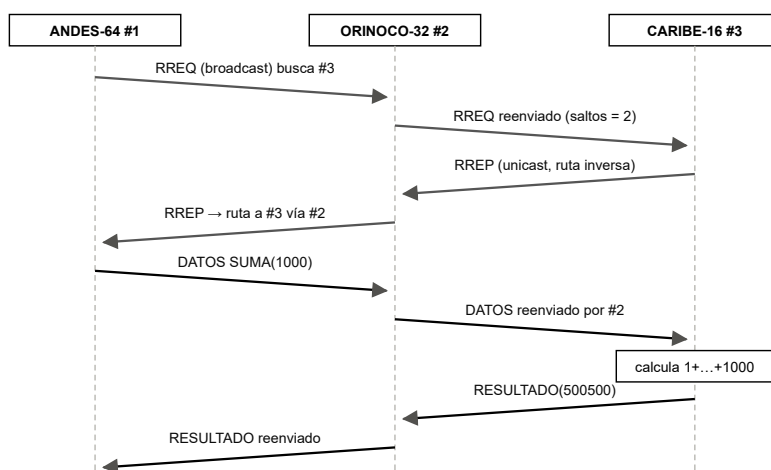


Figura 4. Diagrama de secuencia del escenario 1: descubrimiento de ruta de dos saltos y clculo remoto.

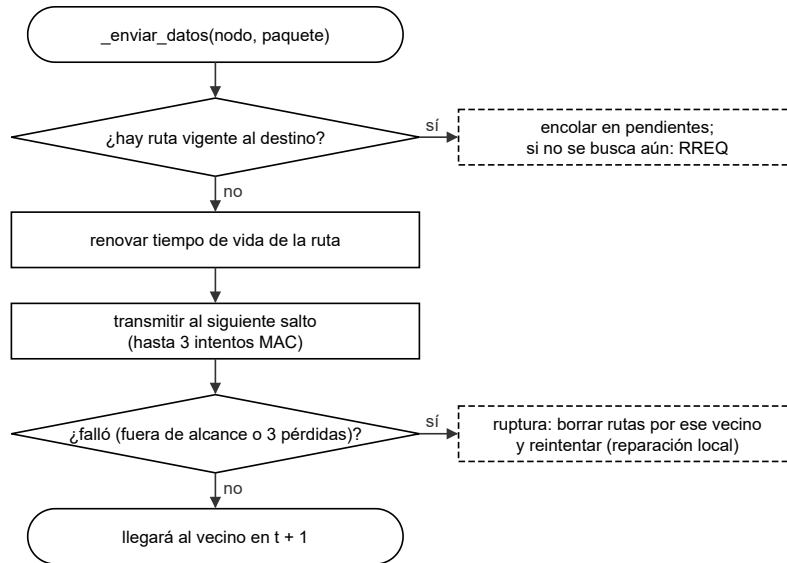


Figura 5. Envío de un paquete de datos desde un nodo.

```

_recibir(nodo, emisor, p):
  si p es RREQ:    si (p.origen, p.id) ya visto: descartar
                  instalar ruta inversa origen → emisor (saltos + 1)
                  si nodo = p.destino: enviar RREP por la ruta inversa
                  si no y saltos < TTL: retransmitir por broadcast
  si p es RREP:    instalar ruta al origen del RREP → emisor
                  si nodo = destino del RREP: enviar los paquetes pendientes
                  si no: reenviar el RREP por la ruta inversa
  si p es DATOS:   instalar ruta de vuelta al origen
                  si nodo = p.destino: NIC.rx ← [origen] + datos ; registrar
latencia          si no: _enviar_datos(nodo, p)           ; el nodo actúa como
enrutador
  
```

4.3 Movilidad

Random waypoint en un área rectangular: destino uniforme, velocidad v (m/s) constante, pausa de 100 ticks (1 s) al llegar. Con $v = 0$ los nodos quedan fijos en posiciones aleatorias.

4.4 Ciclo de simulación

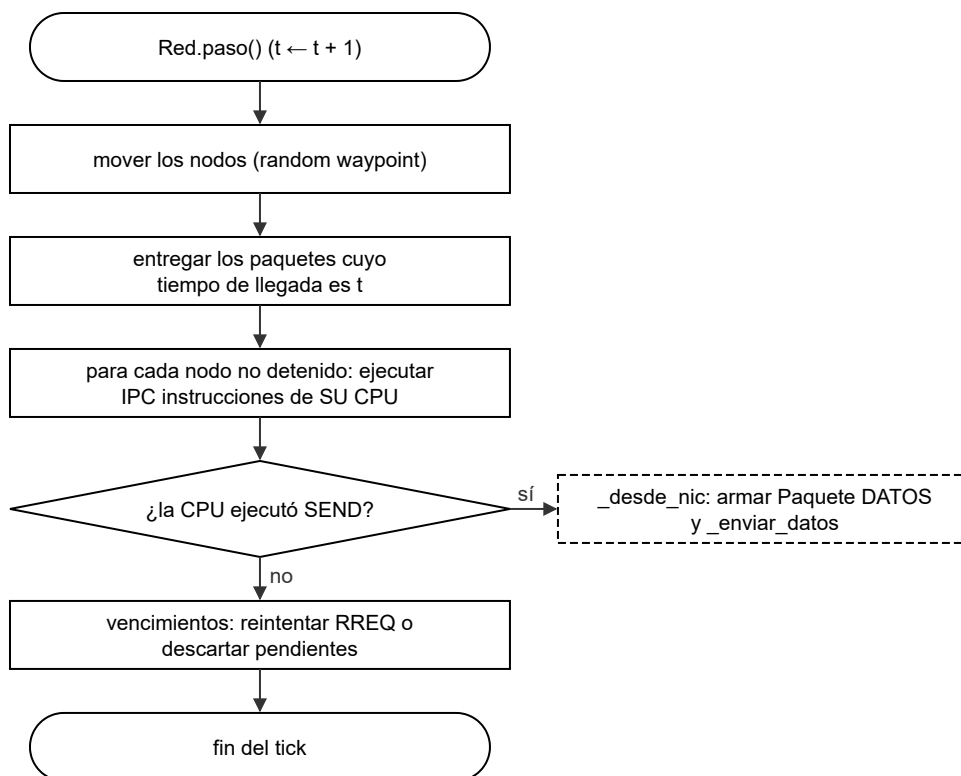


Figura 6. Un tick de simulación. Las CPU se intercalan: en cada tick cada una ejecuta tantas instrucciones como indica su reloj (8, 5 o 3).

5. Programas de los nodos

Programa	Máquina	Función
servidor.a64 / servidor.o32 / servidor.cl6	las tres	Bucle infinito: sondea la NIC; responde PING con PONG y SUMA(n) con RESULTADO(1+...+n). El CARIBE-16 suma en doble precisión (32 bits) con ADC.
cliente_ping.a64	ANDES-64	Lee destino, K, ESPERA y PAUSA; envía K pings; espera cada PONG (descarta los atrasados) e imprime cuántos volvieron.
coordinador.a64	ANDES-64	Lee m parejas (nodo, n), envía m solicitudes SUMA, recoge los resultados en el orden de llegada e imprime el total.
pdu.a64	ANDES-64	Biblioteca: LEER_VALOR, ESCRIBIR_VALOR, PONER_BYTE, IMPRIMIR.

6. Diseño del software

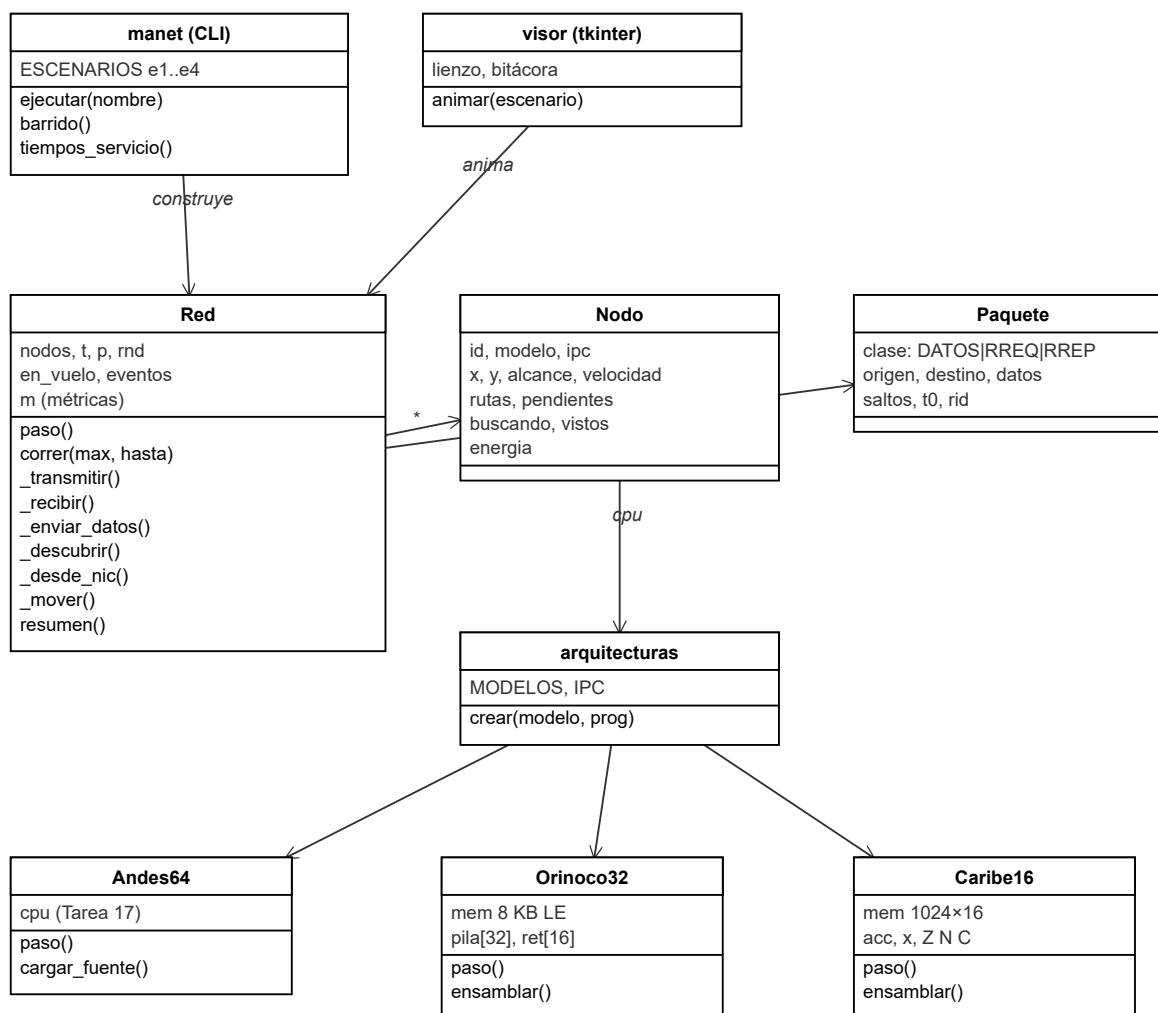


Figura 7. Diagrama de clases/módulos.

Módulo	Responsabilidad
andes64/	CPU, ensamblador y cargador del ANDES-64 (copiados de la Tarea 17).
orinoco32.py	CPU de pila y su ensamblador (sintaxis tipo Forth).
caribe16.py	CPU de acumulador y su ensamblador.
arquitecturas.py	Adaptadores con interfaz común (paso, detenida, enviar, rx), relojes (IPC) y creación de nodos.
red.py	Clases Red, Nodo, Paquete: radio, enlace, AODV, movilidad, métricas y bitácora.
manet.py	Escenarios e1–e4, línea de comandos y barrido de experimentos.
visor.py	Animación en tkinter: nodos, alcances, enlaces, rutas activas y paquetes en vuelo.

(D) CÓDIGO FUENTE COMPLETO

Se transcribe a continuación el código fuente completo de la herramienta construida. Todo está escrito en Python 3 usando únicamente la biblioteca estándar (no hay librerías externas ni versiones adicionales que declarar) y en los lenguajes ensambladores de las máquinas. Los mismos archivos se entregan en la carpeta `codigo/`; los datos generados por los experimentos (`resultados/*.json`) no se transcriben.

Estructura de archivos

<code>codigo/arquitecturas.py</code>	85 líneas
<code>codigo/caribel16.py</code>	260 líneas
<code>codigo/manet.py</code>	173 líneas
<code>codigo/orinoco32.py</code>	268 líneas
<code>codigo/red.py</code>	339 líneas
<code>codigo/visor.py</code>	90 líneas
<code>codigo/andes64/cargador.py</code>	98 líneas
<code>codigo/andes64/ensamblador.py</code>	336 líneas
<code>codigo/andes64/isa.py</code>	158 líneas
<code>codigo/andes64/maquina.py</code>	429 líneas
<code>codigo/programas/andes-64/cliente_ping.a64</code>	58 líneas
<code>codigo/programas/andes-64/coordinador.a64</code>	68 líneas
<code>codigo/programas/andes-64/pdu.a64</code>	53 líneas
<code>codigo/programas/andes-64/servidor.a64</code>	37 líneas
<code>codigo/programas/caribe-16/servidor.c16</code>	63 líneas
<code>codigo/programas/orinoco-32/servidor.o32</code>	29 líneas

`codigo/arquitecturas.py`

```
# -*- coding: utf-8 -*-
"""Adaptadores: presentan a las tres CPU heterogéneas con una interfaz común para la red.

Interfaz común de un nodo-computador:
    cargar(ruta_fuente, entradas)    ensambla con SU propio ensamblador y carga el binario
    paso()                          ejecuta una instrucción de SU propio repertorio
    detenida, instrucciones, salida
    enviar (lo asigna la red)        función que recibe los bytes que la NIC transmite
    rx                               cola de paquetes recibidos que la NIC entrega a la CPU
"""

import os
import sys

AQUI = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.join(AQUI, 'andes64'))

import cargador as _andes_cargador      # noqa: E402
import ensamblador as _andes_ensamblador # noqa: E402
import maquina as _andes                # noqa: E402
from caribel16 import Caribel16         # noqa: E402,F401
from orinoco32 import Orinoco32         # noqa: E402,F401
```

```

class Andes64:
    FABRICANTE = 'Industrias Andinas de Cómputo (IAC)'
    MODELO = 'ANDES-64'
    PALABRA = 64

    def __init__(self):
        self.cpu = _andes.CPU(eco=False)

    @property
    def enviar(self):
        return self.cpu.nic.enlace

    @enviar.setter
    def enviar(self, f):
        self.cpu.nic.enlace = f

    @property
    def rx(self):
        return self.cpu.nic.recibidas

    @property
    def detenida(self):
        return self.cpu.detenida

    @property
    def instrucciones(self):
        return self.cpu.instrucciones

    @property
    def salida(self):
        return self.cpu.es.salida

    def paso(self):
        return self.cpu.paso()

    def estado(self):
        return 'PC=%03X R0=%X R2=%X' % (self.cpu.pc, self.cpu.R[0], self.cpu.R[2])

    def cargar_fuente(self, ruta):
        obj = _andes_ensamblador.ensamblar(ruta)
        _andes_cargador.cargar(self.cpu, obj)
        return obj.tamano()

MODELOS = {'ANDES-64': Andes64, 'ORINOCO-32': Orinoco32, 'CARIBE-16': Caribe16}
EXTENSION = {'ANDES-64': '.a64', 'ORINOCO-32': '.o32', 'CARIBE-16': '.c16'}
# instrucciones que ejecuta cada CPU por tick de simulación (relojes distintos)
IPC = {'ANDES-64': 8, 'ORINOCO-32': 5, 'CARIBE-16': 3}

def crear(modelo, programa, entradas=None):
    """Crea la CPU del modelo, ensambla `programa` (nombre sin extensión dentro de
    programas/<modelo>/) y lo carga. `entradas` alimenta el periférico de entrada."""
    cpu = MODELOS[modelo]()
    ruta = os.path.join(AQUI, 'programas', modelo.lower(), programa + EXTENSION[modelo])
    cpu.cargar_fuente(ruta)
    if entradas:
        if modelo != 'ANDES-64':
            raise ValueError('solo ANDES-64 tiene periférico de entrada de datos')
        cpu.cpu.es.entrada.extend(entradas)
    return cpu

```

codigo/caribel16.py

```
# -*- coding: utf-8 -*-
"""CARIBE-16 (fabricante: Compañía Caribeña de Calculadoras, "CCC")

Máquina de acumulador, palabra de 16 bits, memoria de 1024 palabras direccionable por
palabra (bus de direcciones de 10 bits). Instrucción de longitud fija de 16 bits:

    15      11 10 9      0
+-----+-----+-----+
| opcode | M |      operando      |      M = 0 directo, M = 1 inmediato
+-----+-----+-----+

Registros: ACC (16), X índice (16), PC (10), IR (16). Banderas: Z, N, C.
No tiene multiplicación ni pila: las subrutinas se llaman con JSR (guarda el retorno en la
primera palabra de la subrutina, como el PDP-8) y se retorna con JMI (salto indirecto).
NIC de 8 bits: transmite el byte bajo de cada palabra del búfer.

Lenguaje ensamblador (syntaxis propia de la CCC):
    etiqueta: LDA #5      ; inmediato
              ADD var      ; directo
              LDAX tabla   ; indexado: M[tabla + X]
              ORG 100 / WORD 7 / SPACE 10
"""

import re
from collections import deque

M16 = 0xFFFF
TAM = 1024

# nemonico: (opcode, admite_inmediato)
ISA = {
    'HLT': (0, False), 'LDA': (1, True), 'STA': (2, False), 'ADD': (3, True),
    'SUB': (4, True), 'AND': (5, True), 'OR': (6, True), 'XOR': (7, True),
    'SHL': (8, True), 'SHR': (9, True), 'LDX': (10, True), 'STX': (11, False),
    'ADX': (12, True), 'LDAX': (13, False), 'STAX': (14, False), 'JMP': (15, False),
    'JZ': (16, False), 'JNZ': (17, False), 'JN': (18, False), 'JC': (19, False),
    'JSR': (20, False), 'JMI': (21, False), 'CMP': (22, True), 'IN': (23, False),
    'OUT': (24, True), 'NSND': (25, False), 'NRCV': (26, False), 'NOP': (27, False),
    'ADC': (28, True), 'CPX': (29, True), 'TAX': (30, False), 'TXA': (31, False),
}

SIN_OPERANDO = {'HLT', 'NOP', 'TAX', 'TXA', 'IN'}
POR_CODIGO = {c: n for n, (c, _) in ISA.items()}

class ErrorCaribe(Exception):
    pass

class Caribel16:
    FABRICANTE = 'Compañía Caribeña de Calculadoras (CCC)'
    MODELO = 'CARIBE-16'
    PALABRA = 16

    def __init__(self):
        self.mem = [0] * TAM
        self.salida = []
        self.enviar = None # lo conecta la red
        self.rx = deque()
        self.reiniciar()

    def reiniciar(self, pc=0):
        self.acc = self.x = 0
        self.pc, self.ir = pc, 0
        self.Z = self.N = self.C = 0
        self.detenido = False
```

```

        self.instrucciones = 0

def _banderas(self, v):
    self.Z = 1 if v == 0 else 0
    self.N = (v >> 15) & 1

def paso(self):
    if self.detenida:
        return False
    self.ir = w = self.mem[self.pc]
    op, m, arg = w >> 11, (w >> 10) & 1, w & 0x3FF
    self.pc = (self.pc + 1) % TAM
    self.instrucciones += 1
    nombre = POR_CODIGO[op]
    val = arg if m else self.mem[arg]
    if nombre == 'HLT':
        self.detenida = True
    elif nombre == 'LDA':
        self.acc = val
        self._banderas(self.acc)
    elif nombre == 'STA':
        self.mem[arg] = self.acc
    elif nombre in ('ADD', 'ADC'):
        t = self.acc + val + (self.C if nombre == 'ADC' else 0)
        self.C = 1 if t > M16 else 0
        self.acc = t & M16
        self._banderas(self.acc)
    elif nombre in ('SUB', 'CMP'):
        t = self.acc - val
        self.C = 1 if t < 0 else 0
        self._banderas(t & M16)
        if nombre == 'SUB':
            self.acc = t & M16
    elif nombre == 'AND':
        self.acc &= val
        self._banderas(self.acc)
    elif nombre == 'OR':
        self.acc |= val
        self._banderas(self.acc)
    elif nombre == 'XOR':
        self.acc ^= val
        self._banderas(self.acc)
    elif nombre == 'SHL':
        self.C = (self.acc >> (16 - val)) & 1 if 0 < val <= 16 else 0
        self.acc = (self.acc << val) & M16
        self._banderas(self.acc)
    elif nombre == 'SHR':
        self.C = (self.acc >> (val - 1)) & 1 if val > 0 else 0
        self.acc >>= val
        self._banderas(self.acc)
    elif nombre == 'LDX':
        self.x = val
    elif nombre == 'STX':
        self.mem[arg] = self.x
    elif nombre == 'ADX':
        self.x = (self.x + val) & M16
    elif nombre == 'CPX':
        t = self.x - val
        self.C = 1 if t < 0 else 0
        self._banderas(t & M16)
    elif nombre == 'LDAX':
        self.acc = self.mem[(arg + self.x) % TAM]
        self._banderas(self.acc)
    elif nombre == 'STAX':
        self.mem[(arg + self.x) % TAM] = self.acc
    elif nombre == 'JMP':
        self.pc = arg

```

```

elif nombre == 'JZ':
    self.pc = arg if self.Z else self.pc
elif nombre == 'JNZ':
    self.pc = arg if not self.Z else self.pc
elif nombre == 'JN':
    self.pc = arg if self.N else self.pc
elif nombre == 'JC':
    self.pc = arg if self.C else self.pc
elif nombre == 'JSR':
    self.mem[arg] = self.pc
    self.pc = (arg + 1) % TAM
elif nombre == 'JMI':
    self.pc = self.mem[arg] % TAM
elif nombre == 'TAX':
    self.x = self.acc
elif nombre == 'TXA':
    self.acc = self.x
    self._banderas(self.acc)
elif nombre == 'OUT':
    v = self.acc
    self.salida.append(chr(v & 0xFF) if val == 1 else str(v - 0x10000 if v & 0x8000
else v))
elif nombre == 'IN':
    self.acc = 0
elif nombre == 'NSND':
    n = self.acc
    datos = bytes(self.mem[(arg + i) % TAM] & 0xFF for i in range(n))
    if self.enviar:
        self.enviar(datos)
elif nombre == 'NRCV':
    if self.rx:
        datos = self.rx.popleft()
        for i, b in enumerate(datos):
            self.mem[(arg + i) % TAM] = b
        self.acc = len(datos)
    else:
        self.acc = 0
    self._banderas(self.acc)
return not self.detenida

def estado(self):
    return 'ACC=%04X X=%04X PC=%03X Z=%d N=%d C=%d' % (self.acc, self.x, self.pc,
self.Z, self.N, self.C)

# ----- ensamblador
def cargar_fuente(self, ruta):
    codigo, simbolos = ensamblar(ruta)
    for d, w in codigo.items():
        self.mem[d] = w
    self.reiniciar(simbolos.get('inicio', 0))
    return len(codigo)

def ensamblar(ruta):
    lineas = []
    with open(ruta, encoding='utf-8') as f:
        for num, cruda in enumerate(f, 1):
            t = cruda.split('; ', 1)[0].strip()
            if t:
                lineas.append((num, t))
    simbolos, pc = {}, 0

    def partir(t):
        etq = None
        m = re.match(r'^(\w+):\s*(.*)$', t)
        if m:
            etq, t = m.group(1), m.group(2)

```

```

        p = t.split(None, 1)
        return etq, (p[0].upper() if p else None), (p[1].strip() if len(p) > 1 else '')

for num, t in lineas:
    # pasada 1
    etq, nem, arg = partir(t)
    if nem == 'EQU':
        simbolos[etq] = _num(arg, simbolos)
        continue
    if etq:
        simbolos[etq] = pc
    if nem == 'ORG':
        pc = _num(arg, simbolos)
    elif nem == 'SPACE':
        pc += _num(arg, simbolos)
    elif nem:
        pc += 1
codigo, pc = {}, 0
for num, t in lineas:
    # pasada 2
    etq, nem, arg = partir(t)
    try:
        if nem is None or nem == 'EQU':
            continue
        if nem == 'ORG':
            pc = _num(arg, simbolos)
            continue
        if nem == 'SPACE':
            pc += _num(arg, simbolos)
            continue
        if nem == 'WORD':
            codigo[pc] = _num(arg, simbolos) & M16
        else:
            if nem not in ISA:
                raise ErrorCaribe('nemónico desconocido %s' % nem)
            op, admite = ISA[nem]
            m = 0
            if nem in SIN_OPERANDO:
                v = 0
            else:
                if arg.startswith('#'):
                    if not admite:
                        raise ErrorCaribe('%s no admite modo inmediato' % nem)
                    m, arg = 1, arg[1:]
                v = _num(arg, simbolos)
                if not 0 <= v < TAM:
                    raise ErrorCaribe('operando fuera de rango (0..1023): %d' % v)
            codigo[pc] = (op << 11) | (m << 10) | v
            pc += 1
    except (ErrorCaribe, KeyError, ValueError) as e:
        raise ErrorCaribe('%s:%d: %s' % (ruta, num, e))
return codigo, simbolos

def _num(t, simbolos):
    total = 0
    for signo, tok in re.findall(r'([+-]?)\s*([^\s-]+)', t):
        if tok.startswith('"'):
            v = ord(tok[1])
        elif re.fullmatch(r'0x[0-9a-fA-F]+|\d+', tok):
            v = int(tok, 0)
        else:
            v = simbolos[tok]
        total += -v if signo == '-' else v
    return total

```

codigo/manet.py

```
# -*- coding: utf-8 -*-
"""Emulador de una MANET heterogénea de computadores von Neumann.

python manet.py e1 [-v]                                cálculo distribuido multi-salto (3
fabricantes)
python manet.py e2 [--velocidad 5] [--semilla 1] [-v]    movilidad (random waypoint)
python manet.py e3 [--perdida 0.2] [-v]                 enlaces con pérdidas, cadena de 4 saltos
python manet.py e4 [-v]                                 tiempo de servicio de cada arquitectura
python manet.py visor e2 [...]                           animación en tkinter (misma simulación)
python manet.py barrido                                  ejecuta todos los experimentos del informe
"""

import argparse
import json
import os
import statistics
import sys

from red import Nodo, Red

def e1(verboso=False, **_):
    """Cadena fija 1-2-3: cálculo distribuido multi-salto entre los tres fabricantes.
    El ANDES-64 (#1) no alcanza al CARIBE-16 (#3); el ORINOCO-32 (#2) hace de
    enrutador intermedio y a la vez atiende peticiones."""
    red = Red(ancho=250, alto=60, verboso=verboso)
    red.agregar(Nodo(1, 'ANDES-64', 'coordinador', 10, 30, entradas=[2, 2, 10, 3, 1000]))
    red.agregar(Nodo(2, 'ORINOCO-32', 'servidor', 100, 30))
    red.agregar(Nodo(3, 'CARIBE-16', 'servidor', 190, 30))
    return red, 1, 20000

def e2(velocidad=5.0, semilla=1, pings=40, verboso=False, **_):
    """Movilidad: 6 nodos (2 por fabricante), random waypoint en 300 m x 300 m.
    Alcance de 100 m. El ANDES-64 #1 hace PING al CARIBE-16 #6."""
    red = Red(ancho=300, alto=300, semilla=semilla, verboso=verboso)
    r = red.rnd
    modelos = ['ANDES-64', 'ORINOCO-32', 'CARIBE-16', 'ANDES-64', 'ORINOCO-32', 'CARIBE-
16']
    for i, m in enumerate(modelos, 1):
        prog = 'cliente_ping' if i == 1 else 'servidor'
        ent = [6, pings, 160, 200] if i == 1 else None
        red.agregar(Nodo(i, m, prog, r.uniform(0, 300), r.uniform(0, 300), entradas=ent,
            velocidad=velocidad, pausa=100))
    return red, 1, 60000

def e3(perdida=0.2, semilla=1, pings=30, verboso=False, **_):
    """Pérdidas: cadena de 5 nodos separados 80 m, 4 saltos de #1 a #5.
    Cada transmisión se pierde con probabilidad `perdida`."""
    red = Red(ancho=400, alto=60, p_perdida=perdida, semilla=semilla, verboso=verboso)
    modelos = ['ANDES-64', 'ORINOCO-32', 'CARIBE-16', 'ANDES-64', 'ORINOCO-32']
    for i, m in enumerate(modelos, 1):
        prog = 'cliente_ping' if i == 1 else 'servidor'
        ent = [5, pings, 160, 200] if i == 1 else None
        red.agregar(Nodo(i, m, prog, 20 + 80 * (i - 1), 30, entradas=ent))
    return red, 1, 60000

def e4(n=300, verboso=False, **_):
    """Heterogeneidad: la misma SUMA(n) a un servidor de cada fabricante, a un salto.
    Compara el tiempo de servicio de cada arquitectura."""
    red = Red(ancho=200, alto=200, verboso=verboso)
    red.agregar(Nodo(1, 'ANDES-64', 'coordinador', 100, 100, entradas=[3, 2, n, 3, n, 4,
n]))
```

```

red.agregar(Nodo(2, 'ORINOCO-32', 'servidor', 160, 100))
red.agregar(Nodo(3, 'CARIBE-16', 'servidor', 50, 60))
red.agregar(Nodo(4, 'ANDES-64', 'servidor', 60, 150))
return red, 1, 40000

ESCENARIOS = {'e1': e1, 'e2': e2, 'e3': e3, 'e4': e4}

def tiempos_servicio(red):
    """Para cada SUMA entregada a un servidor, ticks hasta que ese servidor envía el
    RESULTADO."""
    res = {}
    for t_llega, origen, destino, tipo, valor, _, _ in red.entregas:
        if tipo != 1:
            continue
        for t_env, o, d, tp, v in red.envios:
            if o == destino and d == origen and tp == 2 and t_env >= t_llega:
                res[red.nodos[destino].nombre] = (t_env - t_llega, v)
                break
    return res

def ejecutar(nombre, **kw):
    red, principal, limite = ESCENARIOS[nombre](**kw)
    red.log('escenario %s: %s' % (nombre, (ESCENARIOS[nombre].__doc__ or
    '').strip().split('\n')[0]))
    for n in red.nodos.values():
        red.log(' %-12s %-38s programa=%-13s pos=(%.0f, %.0f) %d instr/tick'
        % (n.nombre, n.cpu.FABRICANTE, n.programa, n.x, n.y, n.ipc))
    red.correr(limite, hasta=principal)
    return red

def imprimir_resumen(red, principal=1):
    s = red.resumen()
    print('\n=== salida del programa del nodo #%d ===' % principal)
    print(''.join(red.nodos[principal].cpu.salida).rstrip())
    print('\n=== métricas de la red ===')
    for k, v in s.items():
        if isinstance(v, float):
            v = round(v, 3)
        print(' %-16s %s' % (k, v))
    ts = tiempos_servicio(red)
    if ts:
        print('\n=== tiempo de servicio de SUMA ===')
        for nombre, (t, v) in ts.items():
            print(' %-12s %6d ticks resultado=%d' % (nombre, t, v))
    errores = [n for n in red.nodos.values() if n.error]
    for n in errores:
        print(' ERROR en %s: %s' % (n.nombre, n.error))

def barrido(salida):
    """Experimentos del informe (varias semillas por punto)."""
    res = {}
    red = ejecutar('e1')
    res['e1'] = {'salida': ''.join(red.nodos[1].cpu.salida), 'resumen': red.resumen(),
    'eventos': red.eventos}
    print('e1 listo')
    res['e2'] = []
    for v in (0.0, 2.0, 5.0, 10.0, 20.0):
        filas = []
        for semilla in range(1, 6):
            red = ejecutar('e2', velocidad=v, semilla=semilla)
            filas.append(dict(red.resumen(),
            salida=''.join(red.nodos[1].cpu.salida).strip()))

```

```

        res['e2'].append({'velocidad': v, 'corridas': filas})
        print('e2 v=%s: PDR media %.3f' % (v, statistics.mean(f['pdr'] for f in filas)))
    res['e3'] = []
    for p in (0.0, 0.1, 0.2, 0.3, 0.4, 0.5):
        filas = []
        for semilla in range(1, 6):
            red = ejecutar('e3', perdida=p, semilla=semilla)
            filas.append(dict(red.resumen(),
salida=''.join(red.nodos[1].cpu.salida).strip()))
            res['e3'].append({'perdida': p, 'corridas': filas})
            print('e3 p=%s: PDR media %.3f' % (p, statistics.mean(f['pdr'] for f in filas)))
    res['e4'] = []
    for n in (10, 100, 300, 1000):
        red = ejecutar('e4', n=n)
        res['e4'].append({'n': n, 'servicio': tiempos_servicio(red),
            'salida': ''.join(red.nodos[1].cpu.salida)})
        print('e4 n=%d listo' % n)
    with open(salida, 'w', encoding='utf-8') as f:
        json.dump(res, f, indent=1, ensure_ascii=False)
    print('resultados guardados en', salida)

def main():
    p = argparse.ArgumentParser(description='Emulador de MANET heterogénea')
    p.add_argument('escenario', choices=list(ESCENARIOS) + ['visor', 'barrido'])
    p.add_argument('extra', nargs='?', help='escenario a animar (con "visor")')
    p.add_argument('-v', '--verboso', action='store_true', help='muestra la bitácora de
eventos')
    p.add_argument('--velocidad', type=float, default=5.0, help='m/s (e2)')
    p.add_argument('--perdida', type=float, default=0.2, help='probabilidad de pérdida
(e3)')
    p.add_argument('--semilla', type=int, default=1)
    p.add_argument('--n', type=int, default=300, help='n de SUMA(n) (e4)')
    p.add_argument('--salida', default=os.path.join('resultados', 'resultados.json'))
    a = p.parse_args()
    kw = dict(velocidad=a.velocidad, perdida=a.perdida, semilla=a.semilla, n=a.n)
    if a.escenario == 'barrido':
        os.makedirs(os.path.dirname(a.salida) or '.', exist_ok=True)
        return barrido(a.salida)
    if a.escenario == 'visor':
        import visor
        return visor.animar(a.extra or 'e2', **kw)
    red = ejecutar(a.escenario, verboso=a.verboso, **kw)
    print(red.mapa())
    imprimir_resumen(red)
    return 0

if __name__ == '__main__':
    sys.exit(main())

```

codigo/orinoco32.py

```

# -*- coding: utf-8 -*-
"""ORINOCO-32 (fabricante: Orinoco Systems S.A.)

Máquina de pila (cero direcciones), palabra de 32 bits, memoria de 8 KB direccionable por
byte y ordenada en little-endian (bus de direcciones de 13 bits). Las operaciones toman sus
operandos de la cima de una pila de datos de 32 entradas implementada en registros
internos;
las direcciones de retorno van a una pila de retorno separada de 16 entradas.

Instrucciones de longitud variable (opcode de 8 bits):
1 byte opcode (la mayoría: ADD, DUP, SEND, ...)
2 bytes opcode + inm8 con signo (PUSHB)
3 bytes opcode + dir16 little-endian (JMP, JZ, JNZ, CALL)

```

```

5 bytes opcode + inm32 little-endian (PUSH)

Registros: PC (13), SP de datos (5), RSP de retorno (4), IR (hasta 40 bits).
Banderas: O (desbordamiento aritmético), E (error de pila).
NIC: SEND ( dir n -- ) transmite n bytes; RECV ( dir -- n ) recibe un paquete (0 si no
hay).

Lenguaje ensamblador de Orinoco Systems (estilo Forth, en minúsculas):
fact: dup pushb 1 gt jz fin ... ; comentario
      .org 0x100 .word 7 .bytes 10
"""

import re
from collections import deque

M32 = 0xFFFFFFFF
TAM = 8192

# nemonico: (opcode, bytes de operando)
ISA = {
    'nop': (0x00, 0), 'halt': (0x01, 0),
    'push': (0x10, 4), 'pushb': (0x11, 1), 'dup': (0x12, 0), 'drop': (0x13, 0),
    'swap': (0x14, 0), 'over': (0x15, 0), 'rot': (0x16, 0),
    'add': (0x20, 0), 'sub': (0x21, 0), 'mul': (0x22, 0), 'div': (0x23, 0),
    'mod': (0x24, 0), 'neg': (0x25, 0), 'and': (0x28, 0), 'or': (0x29, 0),
    'xor': (0x2A, 0), 'not': (0x2B, 0), 'shl': (0x2C, 0), 'shr': (0x2D, 0),
    'eq': (0x30, 0), 'lt': (0x31, 0), 'gt': (0x32, 0),
    'jmp': (0x40, 2), 'jz': (0x41, 2), 'jnz': (0x42, 2), 'call': (0x43, 2), 'ret': (0x44,
0),
    'load': (0x50, 0), 'store': (0x51, 0), 'loadb': (0x52, 0), 'storeb': (0x53, 0),
    'in': (0x60, 0), 'out': (0x61, 0), 'outc': (0x62, 0),
    'send': (0x70, 0), 'recv': (0x71, 0), 'bswap': (0x72, 0),
}
POR_CODIGO = {c: (n, k) for n, (c, k) in ISA.items()}

class ErrorOrinoco(Exception):
    pass

def s32(v):
    return v - (1 << 32) if v & 0x80000000 else v

class Orinoco32:
    FABRICANTE = 'Orinoco Systems S.A.'
    MODELO = 'ORINOCO-32'
    PALABRA = 32

    def __init__(self):
        self.mem = bytearray(TAM)
        self.salida = []
        self.enviar = None
        self.rx = deque()
        self.reiniciar()

    def reiniciar(self, pc=0):
        self.pila, self.ret = [], []
        self.pc, self.ir = pc, 0
        self.O = self.E = 0
        self.detenida = False
        self.instrucciones = 0

    # -- pila -----
    def _push(self, v):
        if len(self.pila) >= 32:
            self.E = 1

```

```

        raise ErrorOrinoco('desbordamiento de la pila de datos')
self.pila.append(v & M32)

def _pop(self):
    if not self.pila:
        self.E = 1
        raise ErrorOrinoco('pila de datos vacía (PC=%04X)' % self.pc)
    return self.pila.pop()

def _l32(self, d):
    return int.from_bytes(self.mem[d % TAM:d % TAM + 4], 'little')

def paso(self):
    if self.detenida:
        return False
    op = self.mem[self.pc]
    if op not in POR_CODIGO:
        raise ErrorOrinoco('opcode inválido %02X en %04X' % (op, self.pc))
    nombre, k = POR_CODIGO[op]
    arg = int.from_bytes(self.mem[self.pc + 1:self.pc + 1 + k], 'little') if k else 0
    self.ir = int.from_bytes(self.mem[self.pc:self.pc + 1 + k], 'big')
    self.pc = (self.pc + 1 + k) % TAM
    self.instrucciones += 1
    P, Q = self._push, self._pop
    if nombre == 'halt':
        self.detenida = True
    elif nombre == 'push':
        P(arg)
    elif nombre == 'pushb':
        P(arg - 256 if arg & 0x80 else arg)
    elif nombre == 'dup':
        v = Q(); P(v); P(v)
    elif nombre == 'drop':
        Q()
    elif nombre == 'swap':
        b, a = Q(), Q(); P(b); P(a)
    elif nombre == 'over':
        b, a = Q(), Q(); P(a); P(b); P(a)
    elif nombre == 'rot':
        c, b, a = Q(), Q(), Q(); P(b); P(c); P(a)
    elif nombre in ('add', 'sub', 'mul', 'div', 'mod', 'and', 'or', 'xor',
                    'shl', 'shr', 'eq', 'lt', 'gt'):
        b, a = Q(), Q()
        sa, sb = s32(a), s32(b)
        if nombre == 'add':
            r = sa + sb
        elif nombre == 'sub':
            r = sa - sb
        elif nombre == 'mul':
            r = sa * sb
        elif nombre in ('div', 'mod'):
            if sb == 0:
                raise ErrorOrinoco('división por cero')
            q = abs(sa) // abs(sb) * (1 if (sa >= 0) == (sb >= 0) else -1)
            r = q if nombre == 'div' else sa - q * sb
        elif nombre == 'and':
            r = a & b
        elif nombre == 'or':
            r = a | b
        elif nombre == 'xor':
            r = a ^ b
        elif nombre == 'shl':
            r = (a << b) & M32
        elif nombre == 'shr':
            r = a >> b
        elif nombre == 'eq':
            r = int(a == b)

```

```

        elif nombre == 'lt':
            r = int(sa < sb)
        else:
            r = int(sa > sb)
        if nombre in ('add', 'sub', 'mul'):
            self.O = 1 if not -2 ** 31 <= r < 2 ** 31 else self.O
        P(r)
    elif nombre == 'neg':
        P(~s32(Q()))
    elif nombre == 'not':
        P(~Q())
    elif nombre == 'jmp':
        self.pc = arg
    elif nombre == 'jz':
        if Q() == 0:
            self.pc = arg
    elif nombre == 'jnz':
        if Q() != 0:
            self.pc = arg
    elif nombre == 'call':
        if len(self.ret) >= 16:
            raise ErrorOrinoco('desbordamiento de la pila de retorno')
        self.ret.append(self.pc)
        self.pc = arg
    elif nombre == 'ret':
        self.pc = self.ret.pop()
    elif nombre == 'load':
        P(self._l32(Q()))
    elif nombre == 'store':
        d, v = Q(), Q()
        self.mem[d % TAM:d % TAM + 4] = (v & M32).to_bytes(4, 'little')
    elif nombre == 'loadb':
        P(self.mem[Q() % TAM])
    elif nombre == 'storeb':
        d, v = Q(), Q()
        self.mem[d % TAM] = v & 0xFF
    elif nombre == 'out':
        self.salida.append(str(s32(Q())))
    elif nombre == 'outc':
        self.salida.append(chr(Q() & 0xFF))
    elif nombre == 'in':
        P(0)
    elif nombre == 'bswap':
        P(int.from_bytes((Q() & M32).to_bytes(4, 'little'), 'big'))
    elif nombre == 'send':
        n, d = Q(), Q()
        if self.enviar:
            self.enviar(bytes(self.mem[d:d + n]))
    elif nombre == 'recv':
        d = Q()
        if self.rx:
            datos = self.rx.popleft()
            self.mem[d:d + len(datos)] = datos
            P(len(datos))
        else:
            P(0)
    return not self.detenido

def estado(self):
    return 'PC=%04X pila=%s O=%d' % (self.pc, [s32(v) for v in self.pila[-4:]], self.O)

def cargar_fuente(self, ruta):
    imagen, simbolos = ensamblar(ruta)
    for d, b in imagen.items():
        self.mem[d] = b
    self.reiniciar(simbolos.get('inicio', 0))
    return len(imagen)

```

```

def ensamblar(ruta):
    tokens = [] # (num_linea, token)
    with open(ruta, encoding='utf-8') as f:
        for num, cruda in enumerate(f, 1):
            for t in cruda.split('; ', 1)[0].split():
                tokens.append((num, t))
    simbolos = {}
    for pasada in (1, 2):
        imagen, pc, i = {}, 0, 0
        while i < len(tokens):
            num, t = tokens[i]
            i += 1
            try:
                if t.endswith(':'):
                    if pasada == 1:
                        simbolos[t[:-1]] = pc
                        continue
                if t == '.org':
                    pc = _valor(tokens[i][1], simbolos, pasada)
                    i += 1
                    continue
                if t == '.word':
                    v = _valor(tokens[i][1], simbolos, pasada)
                    i += 1
                    for k, b in enumerate((v & M32).to_bytes(4, 'little')):
                        imagen[pc + k] = b
                    pc += 4
                    continue
                if t == '.bytes':
                    pc += _valor(tokens[i][1], simbolos, pasada)
                    i += 1
                    continue
                if t not in ISA:
                    raise ErrorOrinoco('instrucción desconocida "%s"' % t)
                op, k = ISA[t]
                imagen[pc] = op
                if k:
                    v = _valor(tokens[i][1], simbolos, pasada)
                    i += 1
                    for j, b in enumerate((v & ((1 << (8 * k)) - 1)).to_bytes(k,
'little')):
                        imagen[pc + 1 + j] = b
                    pc += 1 + k
            except (ErrorOrinoco, IndexError, KeyError, ValueError) as e:
                raise ErrorOrinoco('%s:%d: %s' % (ruta, num, e))
    return imagen, simbolos

def _valor(t, simbolos, pasada):
    if re.fullmatch(r'-?(0x[0-9a-fA-F]+|\d+)', t):
        return int(t, 0)
    if t.startswith('"') and len(t) == 3:
        return ord(t[1])
    if pasada == 1:
        return 0
    return simbolos[t]

```

codigo/red.py

```

# -*- coding: utf-8 -*-
"""Emulación de la MANET: medio inalámbrico, movilidad y enrutamiento.

Modelo (discreto en el tiempo, 1 tick = 10 ms simulados):
    * Radio: dos nodos están enlazados si su distancia es <= min(alcance_i, alcance_j).

```

```

Cada transmisión de un salto tarda 1 tick y se pierde con probabilidad p_perdida.
* Capa de enlace: los envíos unicast se reintentan hasta REINTENTOS_MAC veces
  (como el ACK de 802.11). Los broadcast (RREQ) no se reintentan.
* Movilidad: "random waypoint" (destino aleatorio, velocidad v, pausa) o nodos fijos.
* Enrutamiento: AODV simplificado (bajo demanda):
  RREQ por inundación con identificador (origen, id) y TTL,
  ruta inversa en cada nodo intermedio, RREP unicast por la ruta inversa,
  rutas con tiempo de vida, reparación local cuando un enlace se rompe,
  reintento de descubrimiento y descarte tras MAX_RREQ intentos.
* Paquete de aplicación (PDU canónica, independiente de la arquitectura):
  byte 0 = destino al enviar / origen al recibir, byte 1 = tipo, bytes 2.. = datos
  (enteros de 32 bits con signo en big-endian = "orden de red").
"""

import math
import random
from collections import deque

import arquitecturas

TICK_S = 0.01
REINTENTOS_MAC = 3
TTL = 12
VIDA_RUTA = 300          # ticks
ESPERA_RREP = 40         # ticks antes de reintentar un RREQ
MAX_RREQ = 3
TIPOS = {1: 'SUMA', 2: 'RESULTADO', 3: 'PING', 4: 'PONG'}

class Paquete:
    _n = 0

    def __init__(self, clase, origen, destino, datos=b'', **extra):
        Paquete._n += 1
        self.uid = Paquete._n
        self.clase = clase          # DATOS | RREQ | RREP
        self.origen, self.destino = origen, destino
        self.datos = datos
        self.saltos = 0
        self.t0 = extra.pop('t0', 0)
        self.__dict__.update(extra)

class Nodo:
    def __init__(self, nid, modelo, programa, x, y, alcance=100.0, entradas=None,
                  velocidad=0.0, pausa=0):
        self.id, self.modelo = nid, modelo
        self.cpu = arquitecturas.crear(modelo, programa, entradas)
        self.programa = programa
        self.ipc = arquitecturas.IPC[modelo]
        self.x, self.y = float(x), float(y)
        self.alcance = alcance
        self.velocidad, self.pausa = velocidad, pausa
        self.meta = None
        self.espera = 0
        self.rutas = {}             # destino -> [siguiente, saltos, expira]
        self.pendientes = {}        # destino -> deque de paquetes esperando ruta
        self.buscando = {}          # destino -> (t_ultimo_rreq, intentos)
        self.vistos = set()         # (origen, id) de RREQ ya procesados
        self.rreq_id = 0
        self.energia = 0.0
        self.error = None

    @property
    def nombre(self):
        return '%s#%d' % (self.modelo, self.id)

```

```

class Red:
    def __init__(self, ancho=300, alto=300, p_perdida=0.0, semilla=1, verboso=False):
        self.ancho, self.alto = ancho, alto
        self.p = p_perdida
        self.rnd = random.Random(semilla)
        self.nodos = {}
        self.t = 0
        self.en_vuelo = []          # (t_llegada, receptor, emisor, paquete)
        self.eventos = []
        self.verboso = verboso
        self.m = {'datos_app': 0, 'entregados': 0, 'latencias': [], 'tx_datos': 0,
                  'tx_rreq': 0, 'tx_rrep': 0, 'retransmisiones': 0, 'rupturas': 0,
                  'descubrimientos': 0, 'descartados': 0, 'perdidas_radio': 0}
        self.envios, self.entregas = [], []

    # ----- construcción
    def agregar(self, nodo):
        self.nodos[nodo.id] = nodo
        nodo.cpu.enviar = lambda datos, n=nodo: self._desde_nic(n, datos)
        return nodo

    def log(self, texto):
        linea = '[t=%5d] %s' % (self.t, texto)
        self.eventos.append(linea)
        if self.verboso:
            print(linea)

    # ----- radio
    def distancia(self, a, b):
        return math.hypot(a.x - b.x, a.y - b.y)

    def enlazados(self, a, b):
        return a is not b and self.distancia(a, b) <= min(a.alcance, b.alcance)

    def vecinos(self, n):
        return [o for o in self.nodos.values() if self.enlazados(n, o)]

    def enlaces(self):
        ids = sorted(self.nodos)
        return [(i, j) for k, i in enumerate(ids) for j in ids[k + 1:]]
            if self.enlazados(self.nodos[i], self.nodos[j])]

    def _transmitir(self, emisor, paquete, siguiente=None):
        """siguiente=None: broadcast. Devuelve False si el enlace unicast falló."""
        emisor.energia += 1.0
        clave = {'DATOS': 'tx_datos', 'RREQ': 'tx_rreq', 'RREP': 'tx_rrep'}[paquete.clase]
        self.m[clave] += 1
        if siguiente is None:
            for v in self.vecinos(emisor):
                if self.rnd.random() >= self.p:
                    self.en_vuelo.append((self.t + 1, v.id, emisor.id, paquete))
            else:
                self.m['perdidas_radio'] += 1
            return True
        receptor = self.nodos[siguiente]
        for intento in range(REINTENTOS_MAC):
            if intento:
                self.m['retransmisiones'] += 1
                emisor.energia += 1.0
            if not self.enlazados(emisor, receptor):
                break
            if self.rnd.random() >= self.p:
                self.en_vuelo.append((self.t + 1, receptor.id, emisor.id, paquete))
                return True
            self.m['perdidas_radio'] += 1
        return False

```

```

# ----- AODV
def _ruta(self, nodo, destino):
    r = nodo.rutas.get(destino)
    if r and r[2] >= self.t:
        return r
    nodo.rutas.pop(destino, None)
    return None

def _instalar(self, nodo, destino, siguiente, saltos):
    r = self._ruta(nodo, destino)
    if r is None or saltos <= r[1] or r[0] == siguiente:
        nodo.rutas[destino] = [siguiente, saltos, self.t + VIDA_RUTA]

def _enviar_datos(self, nodo, paquete):
    """Reenvía un paquete de datos desde `nodo` hacia paquete.destino."""
    r = self._ruta(nodo, paquete.destino)
    if r is None:
        nodo.pendientes.setdefault(paquete.destino, deque()).append(paquete)
        if paquete.destino not in nodo.buscando:
            self._descubrir(nodo, paquete.destino, 1)
        return
    r[2] = self.t + VIDA_RUTA
    if not self._transmitir(nodo, paquete, r[0]):
        self.m['rupturas'] += 1
        self.log('%s: se rompió el enlace con #%d (ruta a #%d); reparación local'
                % (nodo.nombre, r[0], paquete.destino))
        for d in [d for d, rr in nodo.rutas.items() if rr[0] == r[0]]:
            del nodo.rutas[d]
        self._enviar_datos(nodo, paquete)

def _descubrir(self, nodo, destino, intento):
    nodo.rreq_id += 1
    nodo.buscando[destino] = (self.t, intento)
    nodo.vistos.add((nodo.id, nodo.rreq_id))
    self.m['descubrimientos'] += 1
    self.log('%s: RREQ #%d buscando ruta a #%d (intento %d)'
            % (nodo.nombre, nodo.rreq_id, destino, intento))
    p = Paquete('RREQ', nodo.id, destino, rid=nodo.rreq_id, t0=self.t)
    self._transmitir(nodo, p)

def _vencimientos(self):
    for nodo in self.nodos.values():
        for destino, (t, intento) in list(nodo.buscando.items()):
            if self._ruta(nodo, destino):
                del nodo.buscando[destino]
            elif self.t - t >= ESPERA_RREP:
                if intento < MAX_RREQ:
                    self._descubrir(nodo, destino, intento + 1)
                else:
                    perdidos = nodo.pendientes.pop(destino, [])
                    self.m['descartados'] += len(perdidos)
                    del nodo.buscando[destino]
                    self.log('%s: sin ruta a #%d, se descartan %d paquete(s)'
                            % (nodo.nombre, destino, len(perdidos)))

def _recibir(self, nodo, emisor, p):
    nodo.energia += 0.5
    if p.clase == 'RREQ':
        if (p.origen, p.rid) in nodo.vistos:
            return
        nodo.vistos.add((p.origen, p.rid))
        saltos = p.saltos + 1
        self._instalar(nodo, p.origen, emisor, saltos) # ruta inversa
        if nodo.id == p.destino:
            self.log('%s: RREQ de #%d llegó en %d salto(s); responde RREP'
                    % (nodo.nombre, p.origen, saltos))

```

```

        rrep = Paquete('RREP', nodo.id, p.origen, saltos_ruta=0, t0=self.t)
        self._rrep(nodo, rrep)
    elif saltos < TTL:
        q = Paquete('RREQ', p.origen, p.destino, rid=p.rid, t0=p.t0)
        q.saltos = saltos
        self._transmitir(nodo, q)
    elif p.clase == 'RREP':
        p.saltos_ruta += 1
        self._instalar(nodo, p.origen, emisor, p.saltos_ruta) # ruta directa
        if nodo.id == p.destino:
            self.log('%s: ruta a #%d establecida vía #%d (%d saltos)'
                    % (nodo.nombre, p.origen, emisor, p.saltos_ruta))
            nodo.buscando.pop(p.origen, None)
            for q in nodo.pendientes.pop(p.origen, []):
                self._enviar_datos(nodo, q)
        else:
            self._rrep(nodo, p)
    else: # DATOS
        p.saltos += 1
        self._instalar(nodo, p.origen, emisor, p.saltos) # aprende la ruta de
vuelta
        if nodo.id == p.destino:
            lat = self.t - p.t0
            self.m['entregados'] += 1
            self.m['latencias'].append(lat)
            tipo = p.datos[0] if p.datos else 0
            valor = int.from_bytes(p.datos[1:5], 'big', signed=True) if len(p.datos) >=
5 else 0
            self.entregas.append((self.t, p.origen, nodo.id, tipo, valor, lat,
p.saltos))
            self.log('%s: recibe %s(%d) de #%d [%d salto(s), %d ticks]'
                    % (nodo.nombre, TIPOS.get(tipo, tipo), valor, p.origen, p.saltos,
lat))
            nodo.cpu.rx.append(bytes([p.origen]) + p.datos)
        else:
            self._enviar_datos(nodo, p)

def _rrep(self, nodo, rrep):
    r = self._ruta(nodo, rrep.destino)
    if r is None or not self._transmitir(nodo, rrep, r[0]):
        self.log('%s: no pudo reenviar el RREP hacia #%d' % (nodo.nombre,
rrep.destino))

# ----- interfaz NIC
def _desde_nic(self, nodo, datos):
    if len(datos) < 2:
        return
    destino = datos[0]
    if destino not in self.nodos:
        self.log('%s: destino #%d inexistente' % (nodo.nombre, destino))
        return
    p = Paquete('DATOS', nodo.id, destino, bytes(datos[1:]), t0=self.t)
    tipo = datos[1]
    valor = int.from_bytes(datos[2:6], 'big', signed=True) if len(datos) >= 6 else 0
    self.m['datos_app'] += 1
    self.envios.append((self.t, nodo.id, destino, tipo, valor))
    self.log('%s: envia %s(%d) a #%d' % (nodo.nombre, TIPOS.get(tipo, tipo), valor,
destino))
    self._enviar_datos(nodo, p)

# ----- movilidad
def _mover(self):
    for n in self.nodos.values():
        if n.velocidad <= 0:
            continue
        if n.espera > 0:
            n.espera -= 1

```

```

        continue
    if n.meta is None:
        n.meta = (self.rnd.uniform(0, self.ancho), self.rnd.uniform(0, self.alto))
    dx, dy = n.meta[0] - n.x, n.meta[1] - n.y
    d = math.hypot(dx, dy)
    paso = n.velocidad * TICK_S
    if d <= paso:
        n.x, n.y = n.meta
        n.meta, n.espera = None, n.pausa
    else:
        n.x += dx / d * paso
        n.y += dy / d * paso

# ----- simulación
def paso(self):
    self.t += 1
    self._mover()
    llegan = [e for e in self.en_vuelo if e[0] <= self.t]
    self.en_vuelo = [e for e in self.en_vuelo if e[0] > self.t]
    for _, rid, eid, p in llegan:
        self._recibir(self.nodos[rid], eid, p)
    for n in self.nodos.values():
        if n.error or n.cpu.detenido:
            continue
        try:
            for _ in range(n.ipc):
                if not n.cpu.paso():
                    self.log('%s: la CPU ejecutó HLT' % n.nombre)
                    break
            except Exception as e:
                # error de la CPU de ese nodo
                n.error = str(e)
                self.log('%s: ERROR de CPU: %s' % (n.nombre, e))
    self._vencimientos()

def correr(self, max_ticks, hasta=None):
    """Avanza hasta max_ticks o hasta que el nodo `hasta` se detenga."""
    while self.t < max_ticks:
        self.paso()
        if hasta is not None and self.nodos[hasta].cpu.detenido:
            break
    return self.t

# ----- reportes
def resumen(self):
    m = self.m
    lat = m['latencias']
    return {
        'ticks': self.t,
        'datos_app': m['datos_app'],
        'entregados': m['entregados'],
        'pdr': m['entregados'] / m['datos_app'] if m['datos_app'] else 0.0,
        'latencia_media': sum(lat) / len(lat) if lat else 0.0,
        'latencia_max': max(lat) if lat else 0,
        'tx_datos': m['tx_datos'], 'tx_rreq': m['tx_rreq'], 'tx_rrep': m['tx_rrep'],
        'retransmisiones': m['retransmisiones'], 'rupturas': m['rupturas'],
        'descubrimientos': m['descubrimientos'], 'descartados': m['descartados'],
        'perdidas_radio': m['perdidas_radio'],
        'sobrecarga': (m['tx_rreq'] + m['tx_rrep']) / max(1, m['tx_datos']),
        'energia': {n.nombre: round(n.energia, 1) for n in self.nodos.values()},
        'instrucciones': {n.nombre: n.cpu.instrucciones for n in self.nodos.values()},
    }

def mapa(self, columnas=60):
    """Instantánea ASCII de las posiciones (número = id del nodo)."""
    filas = max(3, round(columnas * self.alto / self.ancho / 2))
    g = [[' ']] * columnas for _ in range(filas)]
    for n in self.nodos.values():

```

```

        c = min(columnas - 1, int(n.x / self.anchos * (columnas - 1)))
        f = min(filas - 1, int(n.y / self.alto * (filas - 1)))
        g[f][c] = str(n.id % 10)
        borde = '+' + '-' * columnas + '+'
        return '\n'.join([borde] + ['|' + ''.join(r) + '|' for r in g] + [borde])

```

codigo/visor.py

```

# -*- coding: utf-8 -*-
"""Visor gráfico (tkinter, biblioteca estándar) de la simulación de la MANET.

Dibuja cada nodo con el color de su fabricante, su alcance de radio, los enlaces vigentes,
los paquetes en vuelo y las rutas activas hacia el destino de los PING; a la derecha
muestra la bitácora de eventos y la salida del nodo principal.
"""

import tkinter as tk

import manet

COLORES = {'ANDES-64': '#2a78d6', 'ORINOCO-32': '#eb6834', 'CARIBE-16': '#1baf7a'}
CLASES = {'DATOS': '#d62828', 'RREQ': '#8d8d8d', 'RREP': '#7b2cbf'}

def animar(nombre, ticks_por_cuadro=4, **kw):
    red, principal, limite = manet.ESCENARIOS[nombre](**kw)
    raiz = tk.Tk()
    raiz.title('MANET heterogénea - escenario %s' % nombre)
    escala = min(640 / red.anchos, 480 / max(red.alto, 120))
    anchos, alto = int(red.anchos * escala) + 40, int(max(red.alto, 120) * escala) + 40
    lienzo = tk.Canvas(raiz, width=anchos, height=alto, bg='white')
    lienzo.grid(row=0, column=0, rowspan=2)
    bitacora = tk.Text(raiz, width=62, height=26, font=('Consolas', 8))
    bitacora.grid(row=0, column=1, sticky='nsew')
    estado = tk.Label(raiz, anchor='w', justify='left', font=('Consolas', 9))
    estado.grid(row=1, column=1, sticky='nsew')
    visto = [0]
    pausa = [False]

    def xy(n):
        return 20 + n.x * escala, 20 + n.y * escala

    def dibujar():
        lienzo.delete('all')
        for n in red.nodos.values():
            x, y = xy(n)
            r = n.alcance * escala
            lienzo.create_oval(x - r, y - r, x + r, y + r, outline='#e6e6e6')
        for i, j in red.enlaces():
            a, b = xy(red.nodos[i]), xy(red.nodos[j])
            lienzo.create_line(*a, *b, fill='b0b0b0')
        for n in red.nodos.values():
            # rutas activas (siguiente salto)
            for destino, (sig, saltos, expira) in n.rutas.items():
                if expira >= red.t and sig in red.nodos:
                    a, b = xy(n), xy(red.nodos[sig])
                    lienzo.create_line(*a, *b, fill='c9a0dc', width=2, arrow='last')
        for _, rid, eid, p in red.en_vuelo:
            a, b = xy(red.nodos[eid]), xy(red.nodos[rid])
            mx, my = (a[0] + b[0]) / 2, (a[1] + b[1]) / 2
            lienzo.create_rectangle(mx - 4, my - 4, mx + 4, my + 4, fill=CLASES[p.clase],
            outline='')
        for n in red.nodos.values():
            x, y = xy(n)
            borde = 'black' if not n.cpu.detenida else 'aaaaaa'
            lienzo.create_oval(x - 11, y - 11, x + 11, y + 11, fill=COLORES[n.modelo],
            outline=borde, width=2)

```

```

        lienzo.create_text(x, y, text=str(n.id), fill='white', font=('Arial', 9,
'bold'))
        lienzo.create_text(x, y + 19, text=n.modelo, font=('Arial', 7))
        y0 = 12
        for m, c in COLORES.items():
            lienzo.create_oval(ancho - 120, y0 - 5, ancho - 110, y0 + 5, fill=c,
outline='')
            lienzo.create_text(ancho - 105, y0, text=m, anchor='w', font=('Arial', 8))
            y0 += 14
        for k, c in CLASES.items():
            lienzo.create_rectangle(ancho - 120, y0 - 4, ancho - 112, y0 + 4, fill=c,
outline='')
            lienzo.create_text(ancho - 105, y0, text=k, anchor='w', font=('Arial', 8))
            y0 += 14
        for linea in red.eventos[visto[0]:]:
            bitacora.insert('end', linea + '\n')
        visto[0] = len(red.eventos)
        bitacora.see('end')
        s = red.resumen()
        estado.config(text='t = %d ticks (%.2f s)   datos: %d enviados, %d entregados (PDR
%.2f)\n'
                        'RREQ tx=%d RREP tx=%d rupturas=%d retransmisiones=%d\n'
                        'salida #d: %s\n[espacio] pausa'
                        % (red.t, red.t * 0.01, s['datos_app'], s['entregados'], s['pdr'],
s['tx_rreq'], s['tx_rrep'], s['rupturas'], s['retransmisiones'],
principal,
                        ''.join(red.nodos[principal].cpu.salida).strip()[-60:]))

    def cuadro():
        if not pausa[0] and red.t < limite and not red.nodos[principal].cpu.detenido:
            for _ in range(ticks_por_cuadro):
                red.paso()
                dibujar()
            raiz.after(30, cuadro)

    raiz.bind('<space>', lambda e: pausa.__setitem__(0, not pausa[0]))
    dibujar()
    cuadro()
    raiz.mainloop()

```

codigo/andes64/cargador.py

```

# -*- coding: utf-8 -*-
"""Módulo Enlazador-Cargador (parte "Cargador").

Formato del archivo objeto (.obj, texto plano, se puede editar bit a bit):
    ORIGEN 000          dirección para la que fue ensamblado (hex)
    INICIO 000          dirección de la primera instrucción a ejecutar
    SIMBOLO nombre 1A0  tabla de símbolos
    REUBICA 004 F3      campo de dirección que debe corregirse si se carga en otra
dirección
    000: 00000011 00000000 0000...   ; línea fuente
El cargador lee el objeto desde la memoria secundaria (disco), lo reubica si se pide otra
dirección de carga y lo monta en la RAM; finalmente fija el PC en la dirección de inicio.
"""

import re

import isa
from ensamblador import Objeto

def guardar_obj(obj, ruta):
    with open(ruta, 'w', encoding='utf-8') as f:
        f.write('; archivo objeto - computador de 64 bits (Tarea 17)\n')
        f.write('ORIGEN %03X\nINICIO %03X\n' % (obj.origen or 0, obj.inicio))

```

```

        for nombre, d in sorted(obj.simbolos.items(), key=lambda x: x[1]):
            f.write('SIMBOLO %s %03X\n' % (nombre, d))
        for d, tipo in obj.reubicacion:
            f.write('REUBICA %03X %s\n' % (d, tipo))
        for d, datos, fuente in obj.segmentos:
            f.write('%03X: %s ; %s\n' % (d, ' '.join(format(b, '08b') for b in datos),
fuente))

def leer_obj(ruta):
    obj = Objeto()
    with open(ruta, encoding='utf-8') as f:
        for linea in f:
            linea = linea.split(';')[0].strip()
            if not linea:
                continue
            p = linea.split()
            if p[0] == 'ORIGEN':
                obj.origen = int(p[1], 16)
            elif p[0] == 'INICIO':
                obj.inicio = int(p[1], 16)
            elif p[0] == 'SIMBOLO':
                obj.simbolos[p[1]] = int(p[2], 16)
            elif p[0] == 'REUBICA':
                obj.reubicacion.append((int(p[1], 16), p[2]))
            else:
                m = re.match(r'([0-9A-Fa-f]+):\s*([01\s]*)$', linea)
                if not m:
                    raise ValueError('línea de objeto inválida: %s' % linea)
                bits = m.group(2).replace(' ', '')
                datos = int(bits, 2).to_bytes(len(bits) // 8, 'big') if bits else b''
                obj.segmentos.append((int(m.group(1), 16), datos, ''))
    return obj

def _corregir(imagen, base, d, tipo, delta):
    """Suma delta al campo de dirección de la instrucción/dato ubicado en d."""
    i = d - base
    if tipo == 'D64':
        v = int.from_bytes(imagen[i:i + 8], 'big')
        imagen[i:i + 8] = ((v + delta) & ((1 << 64) - 1)).to_bytes(8, 'big')
        return
    n = {'F3': 4, 'F5': 5, 'F6': 5}[tipo]
    w = int.from_bytes(imagen[i:i + n], 'big')
    f = int(tipo[1])
    campo = isa.decodificar(f, w)
    if tipo == 'F6':
        nuevo = (campo['inm'] + delta) & 0xFFFF
        w = (w & ~(0xFFFF << 2)) | (nuevo << 2)
    else:
        despl = 4 if tipo == 'F3' else 6
        nuevo = (campo['dir'] + delta) & 0xFFF
        w = (w & ~(0xFFF << despl)) | (nuevo << despl)
    imagen[i:i + n] = w.to_bytes(n, 'big')

def cargar(cpu, obj, direccion=None):
    """Monta el objeto en la RAM. Si `direccion` difiere del origen, reubica.
    Devuelve (dirección de carga, dirección de inicio)."""
    base = obj.origen or 0
    destino = base if direccion is None else direccion
    delta = destino - base
    fin = obj.fin()
    imagen = bytearray(fin - base)
    for d, datos, _ in obj.segmentos:
        imagen[d - base:d - base + len(datos)] = datos
    if delta:

```

```

        for d, tipo in obj.reubicacion:
            _corregir(imagen, base, d, tipo, delta)
    if destino + len(imagen) > len(cpu.mem.ram):
        raise ValueError('el programa no cabe en la RAM a partir de 0x%03X' % destino)
    cpu.mem.ram[destino:destino + len(imagen)] = imagen
    cpu.predecod.clear()
    inicio = obj.inicio + delta
    cpu.pc = inicio
    return destino, inicio

```

codigo/andes64/ensamblador.py

```

# -*- coding: utf-8 -*-
"""Ensamblador de dos pasadas para el computador de 64 bits.

Sintaxis:
    etiqueta: NEMONICO operandos      ; comentario
Directivas:
    .org expr          fija la dirección de ensamblado
    .equ NOMBRE expr   define una constante
    .dword expr, ...   palabras de 64 bits (enteros o etiquetas)
    .double real, ...  reales IEEE 754 de doble precisión
    .string "texto"    bytes ASCII terminados en 0
    .space n           reserva n bytes en cero
    .include "archivo" ensambla otro módulo en este punto
Operandos de memoria (F5): dir | [dir] (indirecto) | dir(R12) (indexado)
"""

import os
import re
import struct

import isa

MASK64 = (1 << 64) - 1

class ErrorEnsamblado(Exception):
    pass

class Objeto:
    """Resultado del ensamblado: código máquina + tabla de simbolos + reubicación."""

    def __init__(self):
        self.origen = None
        self.inicio = 0
        self.simbolos = {}
        self.segmentos = []          # (direccion, bytes, texto_fuente)
        self.reubicacion = []       # (direccion, tipo)  tipo in F3, F5, F6, D64

    def fin(self):
        return max((d + len(b) for d, b, _ in self.segmentos), default=0)

    def tamano(self):
        return self.fin() - (self.origen or 0)

    def _quitar_comentario(linea):
        fuera, cad = [], False
        for ch in linea:
            if ch == '"':
                cad = not cad
            if ch == ';' and not cad:
                break

```

```

        fuera.append(ch)
    return ''.join(fuera).strip()

def _separar_operandos(texto):
    partes, actual, prof, cad = [], '', 0, False
    for ch in texto:
        if ch == '"':
            cad = not cad
        if ch in '([' and not cad:
            prof += 1
        if ch in ')]' and not cad:
            prof -= 1
        if ch == ',' and prof == 0 and not cad:
            partes.append(actual.strip())
            actual = ''
        else:
            actual += ch
    if actual.strip():
        partes.append(actual.strip())
    return partes

class Ensamblador:
    def __init__(self):
        self.simbolos = {}
        self.constantes = {}

    # ----- expresiones
    def valor(self, expr, pc):
        """Evalúa sumas/restas de números, caracteres, constantes y etiquetas.
        Devuelve (valor, usa_etiqueta)."""
        expr = expr.strip()
        tokens = re.findall(r"'[^']*'|[+-]|[^+\-\s]+", expr)
        total, signo, usa_etq, esperando = 0, 1, False, True
        for t in tokens:
            if t in '+-' and len(t) == 1:
                signo = signo * (-1 if t == '-' else 1)
                continue
            if t.startswith('"'):
                v = ord(t[1])
            elif t == '$':
                v, usa_etq = pc, True
            elif re.fullmatch(r'0[xX][0-9a-fA-F_]+|0[bB][01_]+|\d[\d_]*', t):
                v = int(t.replace('_', ''), 0)
            elif t in self.constantes:
                v = self.constantes[t]
            elif t in self.simbolos:
                v, usa_etq = self.simbolos[t], True
            else:
                raise ErrorEnsamblado('símbolo no definido: %s' % t)
            total += signo * v
            signo, esperando = 1, False
        if esperando:
            raise ErrorEnsamblado('expresión vacía: %r' % expr)
        return total, usa_etq

    @staticmethod
    def registro(t):
        t = t.strip().upper()
        if t not in isa.REGISTROS:
            raise ErrorEnsamblado('registro inválido: %s' % t)
        return isa.REGISTROS[t]

    # ----- lectura
    def _leer_lineas(self, ruta, visitados=None):
        visitados = visitados or []

```

```

ruta = os.path.abspath(ruta)
if ruta in visitados:
    raise ErrorEnsamblado('inclusión circular: %s' % ruta)
with open(ruta, encoding='utf-8') as f:
    for num, cruda in enumerate(f, 1):
        limpia = _quitar_comentario(cruda.rstrip('\n'))
        m = re.match(r'\s*\s*\s*\s*\s*([^\s]+)', limpia, re.I)
        if m:
            otra = os.path.join(os.path.dirname(ruta), m.group(1))
            yield from self._leer_lineas(otra, visitados + [ruta])
        else:
            yield os.path.basename(ruta), num, cruda.rstrip('\n'), limpia

def _tamano(self, nemonico, ops):
    n = nemonico.upper()
    n = isa.ALIAS.get(n, n)
    if n in isa.POR_NOMBRE:
        return isa.BYTES[isa.POR_NOMBRE[n][1]]
    if n == '.DWORD' or n == '.DOUBLE':
        return 8 * len(ops)
    if n == '.STRING':
        return len(self._cadena(ops[0])) + 1
    if n == '.SPACE':
        return self.valor(ops[0], 0)[0]
    raise ErrorEnsamblado('nemónico desconocido: %s' % nemonico)

@staticmethod
def _cadena(t):
    t = t.strip()
    if not (t.startswith('"') and t.endswith('"')):
        raise ErrorEnsamblado('se esperaba una cadena entre comillas')
    return t[1:-1].replace('\n', '\n').replace('\t', '\t').encode('ascii')

def _partir(self, limpia):
    etq = None
    m = re.match(r'^([A-Za-z_][\w]*)\s*:(.*)$', limpia)
    if m:
        etq, limpia = m.group(1), m.group(2).strip()
    if not limpia:
        return etq, None, []
    partes = limpia.split(None, 1)
    return etq, partes[0], _separar_operandos(partes[1]) if len(partes) > 1 else []

# ----- ensamblado
def ensamblar_archivo(self, ruta):
    lineas = list(self._leer_lineas(ruta))
    obj = Objeto()
    # ---- pasada 1: direcciones de las etiquetas
    pc = 0
    for arch, num, cruda, limpia in lineas:
        try:
            etq, nem, ops = self._partir(limpia)
            if etq:
                if etq in self.simbolos:
                    raise ErrorEnsamblado('etiqueta duplicada: %s' % etq)
                self.simbolos[etq] = pc
            if not nem:
                continue
            u = nem.upper()
            if u == '.ORG':
                pc = self.valor(ops[0], pc)[0]
                if etq:
                    self.simbolos[etq] = pc
            elif u == '.EQU':
                nombre, expr = ops[0].split(None, 1) if len(ops) == 1 else (ops[0],
ops[1])
                self.constantes[nombre.strip()] = self.valor(expr, pc)[0]

```

```

        else:
            pc += self._tamano(nem, ops)
        except ErrorEnsamblado as e:
            raise ErrorEnsamblado('%s:%d: %s\n    %s' % (arch, num, e, cruda.strip()))
# ---- pasada 2: generación de código
pc = 0
for arch, num, cruda, limpia in lineas:
    try:
        etq, nem, ops = self._partir(limpia)
        if not nem:
            continue
        u = nem.upper()
        if u == '.ORG':
            pc = self.valor(ops[0], pc)[0]
            if obj.origen is None:
                obj.origen = pc
            continue
        if u == '.EQU':
            continue
        if obj.origen is None:
            obj.origen = pc
        datos, reub = self._generar(u, ops, pc)
        for desp, tipo in reub:
            obj.reubicacion.append((pc + desp, tipo))
        obj.segmentos.append((pc, datos, cruda.strip()))
        pc += len(datos)
    except ErrorEnsamblado as e:
        raise ErrorEnsamblado('%s:%d: %s\n    %s' % (arch, num, e, cruda.strip()))
if obj.fin() > 4096:
    raise ErrorEnsamblado('el programa no cabe en la RAM de 4 KB (termina en 0x%X)'
% obj.fin())
obj.simbolos = dict(self.simbolos)
obj.inicio = obj.origen or 0
return obj

def _generar(self, u, ops, pc):
    if u == '.DWORD':
        datos, reub = b'', []
        for i, o in enumerate(ops):
            v, e = self.valor(o, pc)
            datos += (v & MASK64).to_bytes(8, 'big')
            if e:
                reub.append((8 * i, 'D64'))
        return datos, reub
    if u == '.DOUBLE':
        return b''.join(struct.pack('>d', float(o)) for o in ops), []
    if u == '.STRING':
        return self._cadena(ops[0]) + b'\0', []
    if u == '.SPACE':
        return bytes(self.valor(ops[0], pc)[0]), []
    u = isa.ALIAS.get(u, u)
    opcode, f, _ = isa.POR_NOMBRE[u]
    campos, reub = {}, []

    def exigir(n):
        if len(ops) != n:
            raise ErrorEnsamblado('%s espera %d operando(s)' % (u, n))

    if f == 1:
        exigir(0)
    elif f == 2:
        if u in ('IN', 'OUT'):
            if len(ops) not in (1, 2):
                raise ErrorEnsamblado('%s espera registro[, D|H|C|F]' % u)
            campos['rd'] = self.registro(ops[0])
            campos['modo'] = isa.MODOS_ES[ops[1].strip().upper()] if len(ops) == 2 else

```

0

```

        else:
            exigir(1)
            campos['rd'] = self.registro(ops[0])
    elif f == 3:
        exigir(1)
        v, e = self.valor(ops[0], pc)
        self._rango(v, 0, 0xFFF, 'dirección')
        campos['direccion'] = v
        if e:
            reub.append((0, 'F3'))
    elif f == 4:
        if u == 'MOV':
            exigir(2)
            campos['rd'], campos['rf1'] = self.registro(ops[0]), self.registro(ops[1])
        elif u in ('CMP', 'TEST'):
            exigir(2)
            campos['rf1'], campos['rf2'] = self.registro(ops[0]), self.registro(ops[1])
        else:
            exigir(3)
            campos['rd'], campos['rf1'], campos['rf2'] = (self.registro(o) for o in
ops)

    elif f == 5:
        exigir(2)
        campos['rd'] = self.registro(ops[0])
        t = ops[1].strip()
        m = re.fullmatch(r'(.+)\s*(R12\s*)', t, re.I)
        if t.startswith('(') and t.endswith(')'):
            campos['modo'], t = isa.INDIRECTO, t[1:-1]
        elif m:
            campos['modo'], t = isa.INDEXADO, m.group(1)
        v, e = self.valor(t, pc)
        self._rango(v, 0, 0xFFF, 'dirección')
        campos['direccion'] = v
        if e:
            reub.append((0, 'F5'))
    elif f == 6:
        if u == 'MOVK':
            exigir(3)
            campos['modo'] = self.valor(ops[2], pc)[0]
            self._rango(campos['modo'], 0, 3, 'trozo')
        else:
            exigir(2)
            campos['rd'] = self.registro(ops[0])
            v, e = self.valor(ops[1], pc)
            self._rango(v, -0x8000, 0xFFFF, 'inmediato')
            if v < 0:
                campos['modo'] = 1
                campos['inm'] = v & 0xFFFF
            if e:
                reub.append((0, 'F6'))
    valor, bits = isa.codificar(f, opcode, **campos)
    return isa.a_bytes(valor, bits), reub

    @staticmethod
    def _rango(v, lo, hi, que):
        if not lo <= v <= hi:
            raise ErrorEnsamblado('%s fuera de rango: %d' % (que, v))

def ensamblar(ruta):
    return Ensamblador().ensamblar_archivo(ruta)

def listado(obj):
    """Listado con dirección, bits y línea fuente."""
    lineas = []
    for d, datos, fuente in obj.segmentos:

```

```

        bits = ' '.join(format(b, '08b') for b in datos[:8])
        if len(datos) > 8:
            bits += ' ...(%d B)' % len(datos)
        lineas.append('%03X: %-44s ; %s' % (d, bits, fuente))
    return '\n'.join(lineas)

if __name__ == '__main__':
    import sys
    import cargador
    if len(sys.argv) < 2:
        print('uso: python ensamblador.py programa.asm [salida.obj]')
        sys.exit(1)
    o = ensamblar(sys.argv[1])
    salida = sys.argv[2] if len(sys.argv) > 2 else os.path.splitext(sys.argv[1])[0] +
    '.obj'
    cargador.guardar_obj(o, salida)
    print(listado(o))
    print('\n%d bytes, origen 0x%03X -> %s' % (o.tamano(), o.origen, salida))

```

codigo/andes64/isa.py

```

# -*- coding: utf-8 -*-
"""Conjunto de instrucciones (ISA) del computador de 64 bits diseñado en la Tarea 9.

Formatos de microinstrucción (longitud variable, el opcode de 16 bits va siempre primero):

F1  sin operando           opcode16           16 bits
F2  un registro            opcode16 modo2 reg4    24 bits (22 + 2
relleno)
F3  salto / control        opcode16 dir12         28 bits
F4  registro-registro      opcode16 modos4 rd4 rf1_4 rf2_4    32 bits
F5  acceso a memoria       opcode16 modo2 reg4 dir12    36 bits (34 + 2
relleno)
F6  operando inmediato     opcode16 modo2 rd4 inm16    40 bits (38 + 2
relleno)

La memoria es direccionable por byte, así que cada instrucción ocupa ceil(bits/8) bytes y
el
PC avanza exactamente esa cantidad (los bits de relleno no se interpretan).
El byte alto del opcode coincide con el número de formato: 0x01xx -> F1, ..., 0x06xx -> F6.
"""

BITS = {1: 16, 2: 24, 3: 28, 4: 32, 5: 36, 6: 40}
BYTES = {f: (b + 7) // 8 for f, b in BITS.items()}

# (nemonico, opcode, formato, descripcion)
INSTRUCCIONES = [
    # ----- F1: sin operando -----
    ('NOP', 0x0100, 1, 'No realiza ninguna operación'),
    ('HLT', 0x0101, 1, 'Detiene la CPU'),
    ('RET', 0x0102, 1, 'Retorno de subrutina: PC <- pop()'),
    ('EI', 0x0103, 1, 'Habilita interrupciones (INT_EN <- 1)'),
    ('DI', 0x0104, 1, 'Deshabilita interrupciones (INT_EN <- 0)'),
    ('CLC', 0x0105, 1, 'Borra la bandera de acarreo (C <- 0)'),
    ('STC', 0x0106, 1, 'Activa la bandera de acarreo (C <- 1)'),
    # ----- F2: un registro -----
    ('INC', 0x0200, 2, 'R <- R + 1'),
    ('DEC', 0x0201, 2, 'R <- R - 1'),
    ('NOT', 0x0202, 2, 'R <- ~R (complemento a uno)'),
    ('NEG', 0x0203, 2, 'R <- -R (complemento a dos)'),
    ('CLR', 0x0204, 2, 'R <- 0'),
    ('PUSH', 0x0205, 2, 'SP <- SP-8 ; M[SP] <- R'),
    ('POP', 0x0206, 2, 'R <- M[SP] ; SP <- SP+8'),
    ('IN', 0x0207, 2, 'R <- periférico de entrada (modo: D entero, H hex, C carácter, F
real)'),

```

```

('OUT', 0x0208, 2, 'periférico de salida <- R (modo: D entero, H hex, C carácter, F
real)'),
('JR', 0x0209, 2, 'Salto indirecto por registro: PC <- R'),
# ----- F3: saltos -----
('JMP', 0x0300, 3, 'Salto incondicional'),
('JZ', 0x0301, 3, 'Salta si Z=1 (igual)'),
('JNZ', 0x0302, 3, 'Salta si Z=0 (distinto)'),
('JC', 0x0303, 3, 'Salta si C=1 (acarreo / menor sin signo)'),
('JNC', 0x0304, 3, 'Salta si C=0 (mayor o igual sin signo)'),
('JN', 0x0305, 3, 'Salta si N=1 (negativo)'),
('JNN', 0x0306, 3, 'Salta si N=0 (no negativo)'),
('JV', 0x0307, 3, 'Salta si V=1 (desbordamiento)'),
('JNV', 0x0308, 3, 'Salta si V=0'),
('JL', 0x0309, 3, 'Salta si menor con signo (N != V)'),
('JGE', 0x030A, 3, 'Salta si mayor o igual con signo (N == V)'),
('JG', 0x030B, 3, 'Salta si mayor con signo (Z=0 y N == V)'),
('JLE', 0x030C, 3, 'Salta si menor o igual con signo (Z=1 o N != V)'),
('JA', 0x030D, 3, 'Salta si mayor sin signo (C=0 y Z=0)'),
('JBE', 0x030E, 3, 'Salta si menor o igual sin signo (C=1 o Z=1)'),
('CALL', 0x030F, 3, 'Llamada: push(PC) ; R14 <- PC ; PC <- dir'),
# ----- F4: registro-registro -----
('MOV', 0x0400, 4, 'Rd <- Rf1'),
('ADD', 0x0401, 4, 'Rd <- Rf1 + Rf2'),
('ADC', 0x0402, 4, 'Rd <- Rf1 + Rf2 + C'),
('SUB', 0x0403, 4, 'Rd <- Rf1 - Rf2'),
('SBB', 0x0404, 4, 'Rd <- Rf1 - Rf2 - C'),
('MUL', 0x0405, 4, 'Rd <- (Rf1 * Rf2) mod 2^64'),
('DIV', 0x0406, 4, 'Rd <- Rf1 / Rf2 (entera con signo, trunca)'),
('DIVU', 0x0407, 4, 'Rd <- Rf1 / Rf2 (entera sin signo)'),
('MOD', 0x0408, 4, 'Rd <- Rf1 mod Rf2 (con signo)'),
('MODU', 0x0409, 4, 'Rd <- Rf1 mod Rf2 (sin signo)'),
('AND', 0x040A, 4, 'Rd <- Rf1 AND Rf2'),
('OR', 0x040B, 4, 'Rd <- Rf1 OR Rf2'),
('XOR', 0x040C, 4, 'Rd <- Rf1 XOR Rf2'),
('SHL', 0x040D, 4, 'Rd <- Rf1 << Rf2'),
('SHR', 0x040E, 4, 'Rd <- Rf1 >> Rf2 (lógico)'),
('SAR', 0x040F, 4, 'Rd <- Rf1 >> Rf2 (aritmético)'),
('CMP', 0x0410, 4, 'Banderas <- Rf1 - Rf2'),
('TEST', 0x0411, 4, 'Banderas <- Rf1 AND Rf2'),
('LDR', 0x0412, 4, 'Rd <- M64[Rf1 + Rf2]'),
('STR', 0x0413, 4, 'M64[Rf1 + Rf2] <- Rd'),
('LDRB', 0x0414, 4, 'Rd <- M8[Rf1 + Rf2]'),
('STRB', 0x0415, 4, 'M8[Rf1 + Rf2] <- Rd (byte bajo)'),
# ----- F5: memoria -----
('LOAD', 0x0500, 5, 'R <- M64[dir] (modo: directo, [indirecto], dir(R12)
indexado)'),
('STORE', 0x0501, 5, 'M64[dir] <- R'),
('LEA', 0x0502, 5, 'R <- dirección efectiva'),
('NSEND', 0x0503, 5, 'NIC: envía R bytes desde dir'),
('NRECV', 0x0504, 5, 'NIC: recibe un bloque en dir ; R <- bytes recibidos'),
# ----- F6: inmediato -----
('MOVI', 0x0600, 6, 'Rd <- inm (modo 00 extensión con ceros, 01 con signo)'),
('MOVK', 0x0601, 6, 'Inserta inm en el trozo de 16 bits número modo de Rd'),
('ADDI', 0x0602, 6, 'Rd <- Rd + inm'),
('SUBI', 0x0603, 6, 'Rd <- Rd - inm'),
('MULI', 0x0604, 6, 'Rd <- Rd * inm'),
('ANDI', 0x0605, 6, 'Rd <- Rd AND inm'),
('ORI', 0x0606, 6, 'Rd <- Rd OR inm'),
('XORI', 0x0607, 6, 'Rd <- Rd XOR inm'),
('SHLI', 0x0608, 6, 'Rd <- Rd << inm'),
('SHRI', 0x0609, 6, 'Rd <- Rd >> inm (lógico)'),
('SARI', 0x060A, 6, 'Rd <- Rd >> inm (aritmético)'),
('CMPI', 0x060B, 6, 'Banderas <- Rd - inm'),
]

# Alias aceptados por el ensamblador (no son opcodes nuevos)
ALIAS = {'JE': 'JZ', 'JNE': 'JNZ', 'JB': 'JC', 'JAE': 'JNC'}

```

```

POR_NOMBRE = {n: (op, f, d) for n, op, f, d in INSTRUCCIONES}
POR_OPCODE = {op: (n, f, d) for n, op, f, d in INSTRUCCIONES}

# Modos de los periféricos de E/S (campo modo de F2)
MODOS_ES = {'D': 0, 'H': 1, 'C': 2, 'F': 3}
# Modos de direccionamiento de F5
DIRECTO, INDIRECTO, INDEXADO = 0, 1, 2

REGISTROS = {'R%d' % i: i for i in range(16)}
REGISTROS.update({'SP': 13, 'LR': 14, 'ACC': 15})

def codificar(formato, opcode, modo=0, rd=0, rf1=0, rf2=0, direccion=0, inm=0):
    """Devuelve (entero_con_los_bits, numero_de_bits_utiles) de una instrucción."""
    if formato == 1:
        return opcode, 16
    if formato == 2:
        return (opcode << 8) | (modo << 6) | (rd << 2), 24
    if formato == 3:
        return (opcode << 12) | (direccion & 0xFFF), 28
    if formato == 4:
        return (opcode << 16) | (modo << 12) | (rd << 8) | (rf1 << 4) | rf2, 32
    if formato == 5:
        return (opcode << 20) | (modo << 18) | (rd << 14) | ((direccion & 0xFFF) << 2), 36
    if formato == 6:
        return (opcode << 24) | (modo << 22) | (rd << 18) | ((inm & 0xFFFF) << 2), 40
    raise ValueError('formato desconocido')

def a_bytes(valor, bits):
    """Alinea los bits útiles a la izquierda de ceil(bits/8) bytes (relleno con ceros)."""
    n = (bits + 7) // 8
    return (valor << (n * 8 - bits)).to_bytes(n, 'big')

def decodificar(formato, palabra):
    """palabra: entero con los n bytes de la instrucción (relleno incluido)."""
    if formato == 1:
        return {}
    if formato == 2:
        return {'modo': (palabra >> 6) & 3, 'rd': (palabra >> 2) & 15}
    if formato == 3:
        return {'dir': (palabra >> 4) & 0xFFF}
    if formato == 4:
        return {'modo': (palabra >> 12) & 15, 'rd': (palabra >> 8) & 15,
                'rf1': (palabra >> 4) & 15, 'rf2': palabra & 15}
    if formato == 5:
        return {'modo': (palabra >> 22) & 3, 'rd': (palabra >> 18) & 15,
                'dir': (palabra >> 6) & 0xFFF}
    if formato == 6:
        return {'modo': (palabra >> 22) & 3, 'rd': (palabra >> 18) & 15,
                'inm': (palabra >> 2) & 0xFFFF}
    raise ValueError('formato desconocido')

```

codigo/andes64/maquina.py

```

# -*- coding: utf-8 -*-
"""Emulador del computador de 64 bits (Tareas 9 y 10): CPU, jerarquía de memoria,
periféricos de E/S y tarjeta de red. Solo usa la biblioteca estándar de Python."""

import struct
from collections import deque

import isa

```

```

MASK = (1 << 64) - 1
BIT63 = 1 << 63
TAM_RAM = 4096          # 4 KB, bus de direcciones de 12 bits
CICLOS = {'L1': 1, 'L2': 5, 'RAM': 20}

class ErrorCPU(Exception):
    pass

def con_signo(v):
    return v - (1 << 64) if v & BIT63 else v

def bits_a_real(v):
    return struct.unpack('>d', v.to_bytes(8, 'big'))[0]

def real_a_bits(x):
    return int.from_bytes(struct.pack('>d', x), 'big')

# ----- memoria
class Cache:
    """Caché de correspondencia directa con líneas de 8 bytes (solo lleva etiquetas y estadísticas: los datos siempre están actualizados en RAM, escritura inmediata)."""

    def __init__(self, nombre, lineas):
        self.nombre, self.lineas = nombre, lineas
        self.etiquetas = [None] * lineas
        self.aciertos = self.fallos = 0

    def buscar(self, bloque):
        i = bloque % self.lineas
        if self.etiquetas[i] == bloque:
            self.aciertos += 1
            return True
        self.fallos += 1
        return False

    def llenar(self, bloque):
        self.etiquetas[bloque % self.lineas] = bloque

    def reiniciar(self):
        self.etiquetas = [None] * self.lineas
        self.aciertos = self.fallos = 0

class Memoria:
    """Registros -> L1 (128 B) -> L2 (512 B) -> RAM (4 KB) -> disco (archivos .obj)."""

    def __init__(self):
        self.ram = bytearray(TAM_RAM)
        self.l1 = Cache('L1', 16)      # 16 líneas x 8 B = 128 B
        self.l2 = Cache('L2', 64)      # 64 líneas x 8 B = 512 B
        self.ciclos = 0
        self.al_escribir = None        # aviso para invalidar instrucciones predecodificadas

    def _verificar(self, d, n):
        if d < 0 or d + n > TAM_RAM:
            raise ErrorCPU('acceso fuera de la RAM: 0x%03X (+%d)' % (d, n))

    def _jerarquia(self, d, n):
        for b in range(d >> 3, ((d + n - 1) >> 3) + 1):
            if self.l1.buscar(b):
                self.ciclos += CICLOS['L1']
            elif self.l2.buscar(b):

```

```

        self.ciclos += CICLOS['L2']
        self.l1.llenar(b)
    else:
        self.ciclos += CICLOS['RAM']
        self.l2.llenar(b)
        self.l1.llenar(b)

def leer(self, d, n):
    self._verificar(d, n)
    self._jerarquia(d, n)
    return int.from_bytes(self.ram[d:d + n], 'big')

def escribir(self, d, n, valor):
    self._verificar(d, n)
    self._jerarquia(d, n)
    self.ram[d:d + n] = (valor & ((1 << (8 * n)) - 1)).to_bytes(n, 'big')
    if self.al_escribir:
        self.al_escribir(d, n)

def poner_bit(self, d, bit, valor):
    """Escritura directa de un bit (bit 7 = más significativo del byte)."""
    self._verificar(d, 1)
    if valor:
        self.ram[d] |= (1 << bit)
    else:
        self.ram[d] &= ~(1 << bit) & 0xFF
    if self.al_escribir:
        self.al_escribir(d, 1)

# ----- periféricos
class Perifericos:
    """Periférico de entrada (cola de valores o teclado) y de salida (pantalla/búfer)."""

    def __init__(self, entradas=None, eco=True, interactivo=False):
        self.entrada = deque(entradas or [])
        self.salida = []
        self.eco = eco
        self.interactivo = interactivo

    def leer(self, modo):
        if not self.entrada:
            if not self.interactivo:
                raise ErrorCPU('IN: el periférico de entrada no tiene datos')
            self.entrada.append(input('IN> '))
        t = str(self.entrada.popleft()).strip()
        if modo == 0:
            return int(t, 0) & MASK
        if modo == 1:
            return int(t, 16) & MASK
        if modo == 2:
            return ord(t[0]) if t else 10
        return real_a_bits(float(t))

    def escribir(self, valor, modo):
        if modo == 0:
            s = str(con_signo(valor))
        elif modo == 1:
            s = '%016X' % valor
        elif modo == 2:
            s = chr(valor & 0xFF)
        else:
            s = repr(bits_a_real(valor))
        self.salida.append(s)
        if self.eco:
            print(s, end='', flush=True)

```

```

def texto(self):
    return ''.join(self.salida)

class NIC:
    """Tarjeta de red. Aislada, funciona como lazo local (lo que se envía se puede
    recibir);
    conectada a una red, `enlace` es una función que entrega la trama al medio."""

    def __init__(self):
        self.recibidas = deque()
        self.enviadas = []
        self.enlace = None

    def enviar(self, datos):
        self.enviadas.append(bytes(datos))
        if self.enlace:
            self.enlace(bytes(datos))
        else:
            self.recibidas.append(bytes(datos))

# ----- CPU
class CPU:
    def __init__(self, entradas=None, eco=True, interactivo=False):
        self.mem = Memoria()
        self.mem.al_escribir = self._invalidar
        self.es = Perifericos(entradas, eco, interactivo)
        self.nic = NIC()
        self.reiniciar()

    # -- estado -----
    def reiniciar(self, pc=0):
        self.R = [0] * 16
        self.R[13] = TAM_RAM          # SP: la pila crece hacia abajo desde el tope de la
RAM
        self.pc = pc
        self.Z = self.N = self.C = self.V = 0
        self.I = 0
        self.int_en = 0
        self.ir = 0
        self.detenida = False
        self.instrucciones = 0
        self.predecod = {}
        self.mem.ciclos = 0
        self.mem.l1.reiniciar()
        self.mem.l2.reiniciar()

    @property
    def fr(self):
        return self.Z | (self.N << 1) | (self.C << 2) | (self.V << 3) | (self.I << 4)

    def _invalidar(self, d, n):
        for p in range(d - 4, d + n):
            self.predecod.pop(p, None)

    # -- utilidades de banderas -----
    def _zn(self, r):
        self.Z = 1 if r == 0 else 0
        self.N = r >> 63

    def _suma(self, a, b, c=0):
        t = a + b + c
        r = t & MASK
        self.C = 1 if t > MASK else 0
        self.V = 1 if ~(a ^ b) & (a ^ r) & BIT63 else 0
        self._zn(r)

```

```

        return r

def _resta(self, a, b, c=0):
    r = (a - b - c) & MASK
    self.C = 1 if a < b + c else 0          # préstamo
    self.V = 1 if ((a ^ b) & (a ^ r)) & BIT63 else 0
    self._zn(r)
    return r

def _logica(self, r):
    self.C = self.V = 0
    self._zn(r)
    return r

def _shl(self, a, n):
    if n == 0:
        self._zn(a)
        return a
    r = (a << n) & MASK if n < 64 else 0
    self.C = (a >> (64 - n)) & 1 if n <= 64 else 0
    self.V = 0
    self._zn(r)
    return r

def _shr(self, a, n):
    if n == 0:
        self._zn(a)
        return a
    r = a >> n if n < 64 else 0
    self.C = (a >> (n - 1)) & 1 if n <= 64 else 0
    self.V = 0
    self._zn(r)
    return r

def _sar(self, a, n):
    s = con_signo(a)
    if n == 0:
        self._zn(a)
        return a
    r = (s >> min(n, 63)) & MASK
    self.C = (s >> (min(n, 64) - 1)) & 1
    self.V = 0
    self._zn(r)
    return r

def _cond(self, nombre):
    Z, N, C, V = self.Z, self.N, self.C, self.V
    return {
        'JMP': True, 'CALL': True,
        'JZ': Z, 'JNZ': not Z, 'JC': C, 'JNC': not C, 'JN': N, 'JNN': not N,
        'JV': V, 'JNV': not V, 'JL': N != V, 'JGE': N == V,
        'JG': (not Z) and N == V, 'JLE': Z or N != V,
        'JA': (not C) and (not Z), 'JBE': C or Z,
    }[nombre]

def _push(self, v):
    self.R[13] = (self.R[13] - 8) & MASK
    self.mem.escribir(self.R[13], 8, v)

def _pop(self):
    v = self.mem.leer(self.R[13], 8)
    self.R[13] = (self.R[13] + 8) & MASK
    return v

# -- ciclo de instrucción -----
def buscar_decodificar(self, pc):
    """Fetch + Decode. Lee primero los 16 bits del opcode y con él la longitud."""

```

```

op = self.mem.leer(pc, 2)
if op not in isa.POR_OPCODE:
    raise ErrorCPU('opcode inválido 0x%04X en PC=0x%03X' % (op, pc))
nombre, formato = isa.POR_OPCODE[op]
n = isa.BYTES[formato]
palabra = self.mem.leer(pc, n)
self.ir = palabra
return nombre, formato, n, isa.decodificar(formato, palabra)

def paso(self):
    """Ejecuta una instrucción. Devuelve False si la CPU está detenida."""
    if self.detenido:
        return False
    pc = self.pc
    d = self.predecod.get(pc)
    if d is None:
        d = self.buscar_decodificar(pc)
        self.predecod[pc] = d
    else:
        self.mem._jerarquia(pc, d[2]) # el fetch igualmente pasa por las cachés
    nombre, formato, n, c = d
    self.pc = (pc + n) & 0xFFFF
    self.instrucciones += 1
    self._ejecutar(nombre, formato, c)
    return not self.detenido

def correr(self, max_instrucciones=None):
    limite = max_instrucciones or float('inf')
    while not self.detenido and self.instrucciones < limite:
        self.paso()
    return self.detenido

def _ejecutar(self, op, f, c):
    R = self.R
    if f == 4:
        rd, a, b = c['rd'], R[c['rf1']], R[c['rf2']]
        if op == 'MOV': R[rd] = a
        elif op == 'ADD': R[rd] = self._suma(a, b)
        elif op == 'ADC': R[rd] = self._suma(a, b, self.C)
        elif op == 'SUB': R[rd] = self._resta(a, b)
        elif op == 'SBB': R[rd] = self._resta(a, b, self.C)
        elif op == 'MUL':
            t = a * b
            R[rd] = t & MASK
            self.C = self.V = 1 if t > MASK else 0
            self._zn(R[rd])
        elif op in ('DIV', 'MOD', 'DIVU', 'MODU'):
            if b == 0:
                raise ErrorCPU('división por cero')
            if op in ('DIVU', 'MODU'):
                r = a // b if op == 'DIVU' else a % b
            else:
                sa, sb = con_signo(a), con_signo(b)
                q = abs(sa) // abs(sb)
                q = q if (sa >= 0) == (sb >= 0) else -q
                r = q if op == 'DIV' else sa - q * sb
            R[rd] = self._logica(r & MASK)
        elif op == 'AND': R[rd] = self._logica(a & b)
        elif op == 'OR': R[rd] = self._logica(a | b)
        elif op == 'XOR': R[rd] = self._logica(a ^ b)
        elif op == 'SHL': R[rd] = self._shl(a, b)
        elif op == 'SHR': R[rd] = self._shr(a, b)
        elif op == 'SAR': R[rd] = self._sar(a, b)
        elif op == 'CMP': self._resta(a, b)
        elif op == 'TEST': self._logica(a & b)
        elif op == 'LDR': R[rd] = self.mem.leer((a + b) & MASK, 8)
        elif op == 'STR': self.mem.escribir((a + b) & MASK, 8, R[rd])

```

```

        elif op == 'LDRB': R[rd] = self.mem.leer((a + b) & MASK, 1)
        elif op == 'STRB': self.mem.escribir((a + b) & MASK, 1, R[rd])
    elif f == 6:
        rd, modo, inm = c['rd'], c['modo'], c['inm']
        v = inm | (MASK ^ 0xFFFF) if (modo == 1 and inm & 0x8000) else inm
        a = R[rd]
        if op == 'MOVI': R[rd] = v
        elif op == 'MOVK':
            s = 16 * modo
            R[rd] = (a & ~(0xFFFF << s) & MASK) | (inm << s)
        elif op == 'ADDI': R[rd] = self._suma(a, v)
        elif op == 'SUBI': R[rd] = self._resta(a, v)
        elif op == 'MULI':
            t = a * v
            R[rd] = t & MASK
            self.C = self.V = 1 if t > MASK else 0
            self._zn(R[rd])
        elif op == 'ANDI': R[rd] = self._logica(a & v)
        elif op == 'ORI': R[rd] = self._logica(a | v)
        elif op == 'XORI': R[rd] = self._logica(a ^ v)
        elif op == 'SHLI': R[rd] = self._shl(a, inm)
        elif op == 'SHRI': R[rd] = self._shr(a, inm)
        elif op == 'SARI': R[rd] = self._sar(a, inm)
        elif op == 'CMPI': self._resta(a, v)
    elif f == 3:
        if self._cond(op):
            if op == 'CALL':
                self._push(self.pc)
                R[14] = self.pc
                self.pc = c['dir']
    elif f == 2:
        r, modo = c['rd'], c['modo']
        if op == 'INC': R[r] = self._suma(R[r], 1)
        elif op == 'DEC': R[r] = self._resta(R[r], 1)
        elif op == 'NOT': R[r] = self._logica(~R[r] & MASK)
        elif op == 'NEG': R[r] = self._resta(0, R[r])
        elif op == 'CLR': R[r] = self._logica(0)
        elif op == 'PUSH': self._push(R[r])
        elif op == 'POP': R[r] = self._pop()
        elif op == 'IN': R[r] = self.es.leer(modo)
        elif op == 'OUT': self.es.escribir(R[r], modo)
        elif op == 'JR': self.pc = R[r] & 0xFFFF
    elif f == 5:
        r, modo, d = c['rd'], c['modo'], c['dir']
        if modo == isa.INDIRECTO:
            d = self.mem.leer(d, 8) & 0xFFF
        elif modo == isa.INDEXADO:
            d = (d + R[12]) & 0xFFF
        if op == 'LOAD': R[r] = self.mem.leer(d, 8)
        elif op == 'STORE': self.mem.escribir(d, 8, R[r])
        elif op == 'LEA': R[r] = d
        elif op == 'NSEND':
            n = R[r]
            self.nic.enviar(bytes(self.mem.ram[d:d + n]))
        elif op == 'NRECV':
            if self.nic.recibidas:
                datos = self.nic.recibidas.popleft()
                for i, b in enumerate(datos):
                    self.mem.escribir(d + i, 1, b)
                R[r] = len(datos)
            else:
                R[r] = 0
    else: # F1
        if op == 'HLT': self.detenido = True
        elif op == 'RET': self.pc = self._pop() & 0xFFFF
        elif op == 'EI': self.int_en = 1
        elif op == 'DI': self.int_en = 0

```

```

        elif op == 'CLC': self.C = 0
        elif op == 'STC': self.C = 1
R[13] &= MASK
self.I = 1 if (self.es.entrada or self.nic.recibidas) else 0

# -- presentación -----
def estado(self):
    lineas = []
    for i in range(0, 16, 4):
        lineas.append(' '.join('R%-2d=%016X' % (j, self.R[j]) for j in range(i, i +
4)))
    lineas.append('PC=%03X  IR=%010X  SP=%03X  FR=%s  [Z=%d N=%d C=%d V=%d I=%d]
instr=%d'
                    % (self.pc, self.ir, self.R[13], format(self.fr, '08b'),
                        self.Z, self.N, self.C, self.V, self.I, self.instrucciones))
    return '\n'.join(lineas)

def estadisticas(self):
    l1, l2 = self.mem.l1, self.mem.l2
    return ('instrucciones=%d  ciclos_memoria=%d  L1 %d/%d aciertos  L2 %d/%d aciertos'
            % (self.instrucciones, self.mem.ciclos, l1.aciertos, l1.aciertos +
11.fallos,
                l2.aciertos, l2.aciertos + 12.fallos))

```

codigo/programas/andes-64/cliente_ping.a64

```

; =====
; ANDES-64 - cliente PING
; Entrada: destino K (cantidad de pings)  ESPERA (sondeos)  PAUSA (iteraciones)
; Envia PING(1..K) al destino; tras cada uno sondea la NIC hasta ESPERA veces
; esperando el PONG con el mismo numero. Al final imprime cuantos volvieron.
; =====
        .org 0x000
inicio:   IN      R4, D           ; destino
          IN      R8, D           ; K
          IN      R9, D           ; ESPERA
          IN      R12, D          ; PAUSA
          LEA     R10, buf
          MOVI    R5, 1           ; secuencia
          MOVI    R6, 0           ; PONG recibidos
ciclo:    MOVI    R11, 0
          MOV     R0, R4
          CALL    PONER_BYTE      ; destino
          MOVI    R11, 1
          MOVI    R0, 3
          CALL    PONER_BYTE      ; tipo PING
          MOV     R2, R5
          CALL    ESCRIBIR_VALOR
          MOVI    R0, 6
          NSEND   R0, buf
          MOV     R7, R9
esperar:  NRECV   R0, buf
          CMPI    R0, 0
          JNZ     luego
          DEC     R7
          JNZ     esperar
          JMP     pausa           ; se agoto la espera
llego:    MOVI    R11, 1
          LDRB    R1, R10, R11
          CMPI    R1, 4
          JNZ     esperar
          CALL    LEER_VALOR
          CMP     R2, R5
          JNZ     esperar        ; PONG atrasado de otro ping
          INC     R6
pausa:    MOV     R7, R12

```

```

_p:      DEC    R7
        JNZ    _p
        INC    R5
        CMP    R5, R8
        JLE    ciclo
        LEA    R0, t_res
        CALL   IMPRIMIR
        OUT    R6, D
        MOVI   R0, '/'
        OUT    R0, C
        OUT    R8, D
        MOVI   R0, 10
        OUT    R0, C
        HLT

t_res:   .string "PONG recibidos: "
buf:     .space 16
        .include "pdu.a64"

```

codigo/programas/andes-64/coordinador.a64

```

; =====
; ANDES-64 - coordinador de un calculo distribuido
; Entrada: m, y luego m parejas (nodo, n).
; Envia SUMA(n) a cada nodo, espera los m RESULTADOS (en el orden en que
; lleguen), los imprime y muestra el total.
; =====
        .org 0x000
inicio:  IN      R8, D           ; m
        LEA     R10, buf
        MOVI    R9, 0
enviar:  CMP     R9, R8
        JGE     recibir
        MOVI    R11, 0
        IN      R0, D           ; nodo
        CALL    PONER_BYTE
        MOVI    R11, 1
        MOVI    R0, 1           ; tipo SUMA
        CALL    PONER_BYTE
        IN      R2, D           ; n
        CALL    ESCRIBIR_VALOR
        MOVI    R0, 6
        NSEND   R0, buf
        INC     R9
        JMP     enviar
recibir: MOVI    R9, 0           ; respuestas recibidas
        MOVI    R3, 0           ; total
        MOVI    R7, 0xFFFF     ; sondeos maximos
bucle:  CMP     R9, R8
        JGE     fin
        NRECV   R0, buf
        CMPI    R0, 0
        JNZ     llego
        DEC     R7
        JNZ     bucle
        LEA     R0, t_tiempo
        CALL    IMPRIMIR
        JMP     fin
llego:  MOVI    R11, 1
        LDRB    R1, R10, R11
        CMPI    R1, 2
        JNZ     bucle
        LEA     R0, t_nodo
        CALL    IMPRIMIR
        MOVI    R11, 0
        LDRB    R1, R10, R11

```

```

        OUT    R1, D                ; nodo que respondio
        LEA    R0, t_flecha
        CALL   IMPRIMIR
        CALL   LEER_VALOR
        OUT    R2, D
        MOVI   R0, 10
        OUT    R0, C
        ADD    R3, R3, R2
        INC    R9
        JMP    bucle
fin:     LEA    R0, t_total
        CALL   IMPRIMIR
        OUT    R3, D
        MOVI   R0, 10
        OUT    R0, C
        HLT

t_nodo:  .string "resultado del nodo #"
t_flecha: .string " = "
t_total: .string "total = "
t_tiempo: .string "tiempo agotado\n"
buf:     .space 16
        .include "pdu.a64"

```

codigo/programas/andes-64/pdu.a64

```

; =====
; ANDES-64: rutinas de apoyo para la PDU canonica de la MANET
;   byte 0 destino/origen | byte 1 tipo | bytes 2..5 entero de 32 bits big-endian
;   R10 = direccion base del bufer de la PDU
; =====

; LEER_VALOR: R2 <- entero de 32 bits con signo (orden de red) de R10+2..5
LEER_VALOR: PUSH    R0
            PUSH    R11
            MOVI    R2, 0
            MOVI    R11, 2
_lv_b:     LDRB     R0, R10, R11
            SHLI    R2, 8
            OR      R2, R2, R0
            INC     R11
            CMPI    R11, 6
            JL      _lv_b
            SHLI    R2, 32          ; extension de signo de 32 a 64 bits
            SARI    R2, 32
            POP     R11
            POP     R0
            RET

; ESCRIBIR_VALOR: escribe los 32 bits bajos de R2 en R10+2..5 (big-endian)
ESCRIBIR_VALOR: PUSH R2
            PUSH    R11
            MOVI    R11, 5
_ev_b:     STRB     R2, R10, R11
            SHRI    R2, 8
            DEC     R11
            CMPI    R11, 2
            JGE     _ev_b
            POP     R11
            POP     R2
            RET

; PONER_BYTE: M8[R10 + R11] <- R0
PONER_BYTE: STRB    R0, R10, R11
            RET

```

```

; IMPRIMIR: escribe la cadena terminada en 0 que empieza en R0
IMPRIMIR:  PUSH  R1
           PUSH  R2
           MOVI  R2, 0
_imp_b:    LDRB  R1, R0, R2
           CMPI  R1, 0
           JZ    _imp_f
           OUT   R1, C
           INC   R2
           JMP   _imp_b
_imp_f:    POP   R2
           POP   R1
           RET

```

codigo/programas/andes-64/servidor.a64

```

; =====
; ANDES-64 - servidor de la MANET
;   PING(v) -> responde PONG(v) al origen
;   SUMA(n)  -> responde RESULTADO(1 + 2 + ... + n)
; =====
.org 0x000
inicio:  LEA    R10, buf
espera:  NRECV  R0, buf          ; R0 = bytes recibidos (0 si no hay)
        CMPI  R0, 0
        JZ    espera
        MOVI  R11, 1
        LDRB  R1, R10, R11      ; tipo
        CALL  LEER_VALOR        ; R2 = valor
        CMPI  R1, 3
        JZ    ping
        CMPI  R1, 1
        JNZ   espera           ; tipo desconocido: se ignora
        MOVI  R3, 0             ; suma
        MOVI  R4, 1
suma:    CMP   R4, R2
        JG    fin_suma
        ADD   R3, R3, R4
        INC   R4
        JMP   suma
fin_suma: MOV   R2, R3
        MOVI  R1, 2
        JMP   responder
ping:    MOVI  R1, 4
responder: MOVI R11, 1          ; buf[0] ya contiene el origen, que ahora
        STRB  R1, R10, R11      ; es el destino de la respuesta
        CALL  ESCRIBIR_VALOR
        MOVI  R0, 6
        NSEND R0, buf
        JMP   espera

buf:     .space 16
        .include "pdu.a64"

```

codigo/programas/caribe-16/servidor.c16

```

; =====
; CARIBE-16 - servidor de la MANET (ensamblador de la CCC)
;   PING(v) -> PONG(v)          SUMA(n) -> RESULTADO(1 + ... + n)
;   La NIC de 8 bits deja un byte por palabra en el bufer.
;   Como la palabra es de 16 bits y 1+...+n puede superar 65535, la suma se
;   lleva en DOBLE PRECISION (hi:lo) propagando el acarreo con ADC.
; =====
inicio:

```

```

espera:      NRCV  buf                ; ACC = bytes recibidos
             JZ    espera
             LDA   buf+1              ; tipo
             CMP   #3
             JZ    ping
             CMP   #1
             JNZ   espera
             LDA   buf+4              ; n = buf[4]*256 + buf[5] (16 bits)
             SHL   #8
             OR    buf+5
             STA   n
             LDA   #0
             STA   hi
             STA   lo
             LDA   #1
             STA   i
bucle:       LDA   n                  ; mientras i <= n
             SUB   i
             JC    fin
             LDA   lo                  ; (hi:lo) += i
             ADD   i
             STA   lo
             LDA   hi
             ADC   #0
             STA   hi
             LDA   i
             ADD   #1
             STA   i
             JMP   bucle
fin:         LDA   hi                  ; resultado de 32 bits en orden de red
             SHR   #8
             STA   buf+2
             LDA   hi
             AND   #255
             STA   buf+3
             LDA   lo
             SHR   #8
             STA   buf+4
             LDA   lo
             AND   #255
             STA   buf+5
             LDA   #2
             STA   buf+1
             JMP   enviar
ping:        LDA   #4                  ; PONG: se devuelve el mismo valor
             STA   buf+1
enviar:       LDA   #6
             NSND  buf                ; buf[0] (origen) pasa a ser el destino
             JMP   espera

n:           WORD  0
i:           WORD  0
hi:          WORD  0
lo:          WORD  0
buf:         SPACE 16

```

codigo/programas/orinoco-32/servidor.o32

```

; =====
; ORINOCO-32 - servidor de la MANET (lenguaje de pila de Orinoco Systems)
;   PING(v) -> PONG(v)          SUMA(n) -> RESULTADO(1 + ... + n)
; La memoria es little-endian: los enteros de la PDU (big-endian, orden de
; red) se convierten con bswap al leerlos y al escribirlos.
; =====
inicio:
espera:      push buf recv jz espera      ; ( -- ) sondea la NIC

```

```

push buf pushb 1 add loadb      ; ( tipo )
dup pushb 3 eq jnz es_ping
pushb 1 eq jz espera           ; solo se atiende SUMA
call leer                      ; ( n )
pushb 0 swap                   ; ( suma n )
bucle:  dup jz fin_suma
        dup rot add swap       ; ( suma+n n )
        pushb 1 sub            ; ( suma+n n-1 )
        jmp bucle
fin_suma: drop pushb 2          ; ( suma 2 )
        jmp responder
es_ping: drop call leer pushb 4 ; ( v 4 )
responder: push buf pushb 1 add storeb ; buf[1] <- tipo ( valor )
        bswap push buf pushb 2 add store ; buf[2..5] <- valor en orden de red
        push buf pushb 6 send ; buf[0] ya es el destino (el origen)
        jmp espera

leer:   push buf pushb 2 add load bswap ; ( -- n )
        ret

buf:    .bytes 16

```

(E) MANUAL DE USUARIO Y MANUAL TÉCNICO

Parte I — Manual de usuario

1. Requisitos

- Python 3.8 o superior con la biblioteca estándar (incluye tkinter para el visor). No se instala nada más.
- Todos los comandos se ejecutan dentro de la carpeta `codigo/`.

```
codigo/
manet.py          programa principal (escenarios, barrido de
experimentos, visor)
red.py            medio de radio, enlace, AODV, movilidad, métricas
arquitecturas.py interfaz común de las tres CPU
andes64/          CPU ANDES-64 + ensamblador + cargador
orinoco32.py      CPU ORINOCO-32 + ensamblador
caribel6.py       CPU CARIBE-16 + ensamblador
visor.py          animación gráfica (tkinter)
programas/andes-64/*.a64  programas en ensamblador ANDES-64
programas/orinoco-32/*.o32 programas en ensamblador ORINOCO-32
programas/caribe-16/*.c16 programas en ensamblador CARIBE-16
resultados/resultados.json datos de los experimentos
```

2. Ejecutar un escenario

```
python manet.py e1 [-v]          cálculo distribuido multi-
salto
python manet.py e2 [--velocidad 5] [--semilla 1] [-v]  movilidad
python manet.py e3 [--perdida 0.2] [--semilla 1] [-v]  enlaces con pérdidas
python manet.py e4 [--n 300] [-v]      tiempo de servicio de cada
fabricante
```

Con `-v` se imprime la bitácora completa de eventos (envíos, RREQ, RREP, rupturas, entregas con su número de saltos y latencia). Al final se muestran un mapa ASCII de las posiciones, la salida del programa del nodo #1 y las métricas.

```

> python manet.py e1
...
=== salida del programa del nodo #1 ===
resultado del nodo #2 = 55
resultado del nodo #3 = 500500
total = 500555

=== métricas de la red ===
ticks          4403
datos_app      4
entregados     4
pdr            1.0
latencia_media 4.0
...
=== tiempo de servicio de SUMA ===
ORINOCO-32#2    25 ticks  resultado=55
CARIBE-16#3    4345 ticks resultado=500500

```

Métrica	Significado
ticks	Duración simulada (1 tick = 10 ms).
datos_app / entregados / pdr	Paquetes de datos enviados por las aplicaciones, entregados a su destino, y su cociente.
latencia_media / latencia_max	Ticks entre el envío y la entrega (incluye el descubrimiento de ruta).
tx_datos / tx_rreq / tx_rrep	Transmisiones de radio de cada clase (un paquete de 3 saltos cuenta 3).
retransmisiones, perdidas_radio	Reintentos de la capa de enlace y transmisiones perdidas.
rupturas, descubrimientos, descartados	Enlaces declarados rotos, RREQ originados, paquetes descartados por no hallar ruta.
sobrecarga	$(tx_rreq + tx_rrep) / tx_datos$.
energia, instrucciones	Por nodo: energía gastada (1 por transmisión, 0,5 por recepción) e instrucciones ejecutadas por su CPU.

3. Visor gráfico

```
python manet.py visor e2 --velocidad 10
```

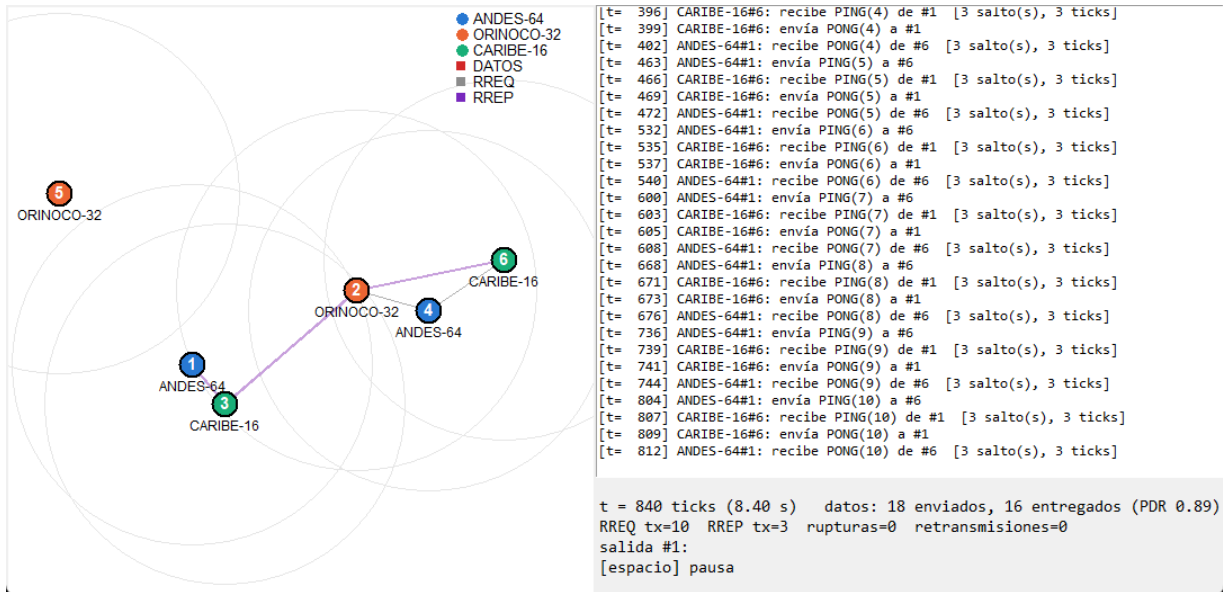


Figura 1. Visor: color por fabricante (azul ANDES-64, naranja ORINOCO-32, verde CARIBE-16), círculos de alcance, enlaces (gris), rutas activas (flechas violeta) y paquetes en vuelo (cuadros rojo DATOS, gris RREQ, violeta RREP). A la derecha, la bitácora y las métricas en vivo. La barra espaciadora pausa.

4. Reproducir todos los experimentos

```
python manet.py barrido # ≈ 10 s; escribe resultados/resultados.json
```

Ejecuta e1; e2 con $v \in \{0, 2, 5, 10, 20\}$ m/s y e3 con $p \in \{0; 0,1; \dots; 0,5\}$, cada uno con las semillas 1 a 5; y e4 con $n \in \{10, 100, 300, 1000\}$. Luego `python docs_src/construir_pdfs.py` (desde la carpeta de la tarea) regenera las gráficas y el PDF único de la entrega.

5. Programar los nodos

5.1 ANDES-64 (.a64)

```
inicio:      .org 0x000
            LEA  R10, buf
            MOVI R11, 0
            MOVI R0, 3          ; destino: nodo #3
            CALL PONER_BYTE
            MOVI R11, 1
            MOVI R0, 3          ; tipo PING
            CALL PONER_BYTE
            MOVI R2, 42
            CALL ESCRIBIR_VALOR ; valor en orden de red
            MOVI R0, 6
            NSEND R0, buf       ; transmite 6 bytes
espera:      NRECV R0, buf       ; R0 = bytes recibidos
            CMPI R0, 0
            JZ   espera
            HLT
buf:         .space 16
            .include "pdu.a64"
```

5.2 ORINOCO-32 (.o32)

Sintaxis en minúsculas, un token por instrucción, operandos a continuación: push 1000, pushb -1, jz etiqueta, call rutina. Etiquetas terminadas en dos puntos. Directivas .org, .word, .bytes. La ejecución empieza en la etiqueta inicio (o en 0). Recuerde que la memoria es little-endian: use bswap al leer o escribir enteros de la PDU.

5.3 CARIBE-16 (.c16)

Una instrucción por línea: LDA #5 (inmediato 0–1023), ADD var (directo), LDAX tabla (indexado por X), JSR sub / JMI sub (subrutinas; la primera palabra de sub se reserva con WORD 0). Directivas ORG, WORD, SPACE, EQU. Las expresiones admiten etiqueta+k.

5.4 Crear un escenario nuevo

```
def e5(verboso=False, **_):
    red = Red(ancho=300, alto=100, p_perdida=0.1, semilla=7, verboso=verboso)
    red.agregar(Nodo(1, 'ANDES-64', 'cliente_ping', 20, 50, entradas=[3, 10,
160, 200]))
    red.agregar(Nodo(2, 'CARIBE-16', 'servidor', 110, 50))
    red.agregar(Nodo(3, 'ORINOCO-32', 'servidor', 200, 50, velocidad=3,
pausa=50))
    return red, 1, 20000          # red, nodo principal, límite de ticks
ESCENARIOS['e5'] = e5
```

Parámetros de `cliente_ping`: destino, cantidad de pings, sondeos de espera por PONG, iteraciones de pausa. De `coordinador`: m y luego m parejas (nodo, n).

Parte II — Manual técnico

6. Interfaz común de las CPU

```
class CPU:                                # ANDES-64 (adaptador), Orinoco32, Caribel6
    FABRICANTE, MODELO, PALABRA
    cargar_fuente(ruta)                    # ensambla con SU ensamblador y carga el
    binario
    paso() -> bool                          # ejecuta UNA instrucción de su ISA
    detenida, instrucciones, salida
    enviar                                  # función asignada por la red: recibe los
    bytes de la NIC
    rx                                      # deque de paquetes entregados por la red
```

La red solo usa esa interfaz; agregar un cuarto fabricante consiste en escribir su clase y registrarla en `arquitecturas.MODELOS`, `EXTENSION` e `IPC`.

7. Flujo de un mensaje

1. El programa ejecuta su instrucción de envío (`NSEND` / `send` / `NSND`). La CPU entrega los bytes a `enviar` → `Red._desde_nic(nodo, datos)`.
2. La red crea un `Paquete('DATOS')` con origen, destino (byte 0) y datos (bytes 1...), registra el envío y llama a `_enviar_datos`.
3. Sin ruta: el paquete queda en `nodo.pendientes` y se inicia `_descubrir` (RREQ). Con ruta: `_transmitir` intenta hasta 3 veces; si tiene éxito, el paquete se agrega a `en_vuelo` con llegada en `t + 1`.
4. En el tick siguiente, `_recibir` en el vecino reenvía o, si es el destino, agrega `[origen] + datos` a `cpu.rx`.
5. El programa del destino lo lee con su instrucción de recepción.

8. Detalles de implementación

- *Determinismo*: toda la aleatoriedad usa `Red.rnd = random.Random(semilla)`; el orden de procesamiento de nodos y paquetes es fijo, por lo que una misma semilla produce exactamente la misma bitácora.
- *Tiempo*: `TICK_S = 0.01`; el movimiento por tick es $v \cdot 0,01$ m.
- *Errores de CPU*: si una CPU falla (opcode inválido, pila vacía...), el nodo se marca con `error`, deja de ejecutar y sigue funcionando como enrutador (la NIC y la capa de red no dependen de la CPU).

- *Parámetros* (en `red.py`): `REINTENTOS_MAC = 3`, `TTL = 12`, `VIDA_RUTA = 300`, `ESPERA_RREP = 40`, `MAX_RREQ = 3`; relojes en `arquitecturas.IPC`.
- *Reutilización*: `andes64/` es una copia sin cambios del emulador de la Tarea 17; su NIC ya tenía el atributo `enlace` para conectarse a una red.

9. Solución de problemas

Síntoma	Causa probable
ErrorOrinoco: pila de datos vacía	Una secuencia del programa .o32 consume más valores de los que apila (revise los comentarios de efecto de pila).
operando fuera de rango (0..1023)	En CARIBE-16 los inmediatos son de 10 bits sin signo.
El cliente imprime 0/K	Destino inalcanzable (red particionada); pruebe otra semilla o más velocidad, y revise con <code>-v</code> .
El visor no abre	La instalación de Python no incluye tkinter (en Linux: paquete <code>python3-tk</code>).

(F) EXPERIMENTACIÓN Y ANÁLISIS DE RESULTADOS

1. Metodología

Se definieron cuatro escenarios (se piden al menos tres). Todos se reproducen con `python manet.py barrido`, que guarda los datos en `resultados/resultados.json`; las tablas y gráficas de este documento se generan desde ese archivo. En los escenarios con aleatoriedad (posiciones, movilidad, pérdidas) cada punto es el promedio de **5 corridas con semillas 1 a 5**, y se muestran también los resultados individuales. Parámetros comunes: alcance de radio 100 m, 1 tick = 10 ms, 1 tick por salto, 3 intentos MAC, relojes de 8 / 5 / 3 instrucciones por tick (ANDES-64 / ORINOCO-32 / CARIBE-16).

Escenario	Pregunta	Variable
E1. Cálculo distribuido multi-salto	¿Cooperan correctamente tres fabricantes distintos, con enrutamiento a través de un nodo de otra arquitectura?	—
E2. Movilidad	¿Cómo afecta la velocidad de los nodos a la entrega y al control?	$v \in \{0, 2, 5, 10, 20\}$ m/s
E3. Enlaces con pérdidas	¿Cuánta pérdida por transmisión tolera un camino de 4 saltos?	$p \in \{0; 0,1; \dots; 0,5\}$
E4. Heterogeneidad de las CPU	¿Cuánto tarda cada arquitectura en atender la misma solicitud?	$n \in \{10, 100, 300, 1000\}$

2. Escenario 1 — Cálculo distribuido multi-salto

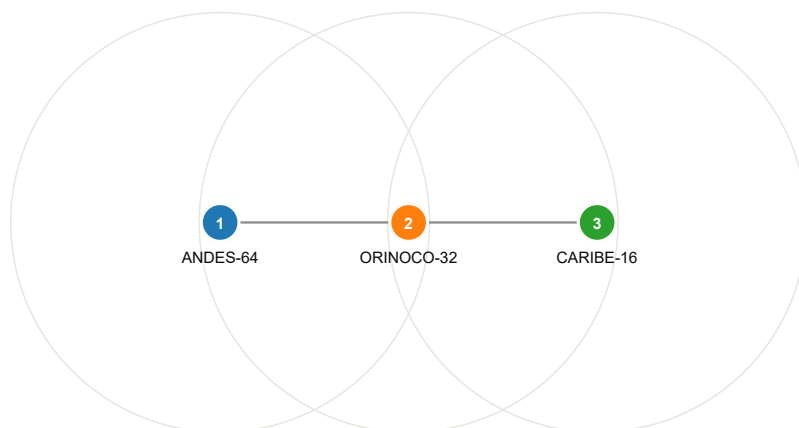


Figura 1. Topología fija: 90 m entre vecinos; #1 y #3 (180 m) no están en alcance.

El coordinador ANDES-64 (#1) pide SUMA(10) al ORINOCO-32 (#2) y SUMA(1000) al CARIBE-16 (#3); este último solo es alcanzable a través del ORINOCO-32, que así funciona a la vez como servidor y como enrutador.

Salida del programa del coordinador

```
resultado del nodo #2 = 55
resultado del nodo #3 = 500500
total = 500555
```

Bitácora de la red

```
[t= 0] escenario e1: Cadena fija 1-2-3: cálculo distribuido multi-salto entre los tres
fabricantes.
[t= 0] ANDES-64#1 Industrias Andinas de Cómputo (IAC) programa=coordinador pos=
(10, 30) 8 instr/tick
[t= 0] ORINOCO-32#2 Orinoco Systems S.A. programa=servidor pos=
(100, 30) 5 instr/tick
[t= 0] CARIBE-16#3 Compañía Caribeña de Calculadoras (CCC) programa=servidor
pos=(190, 30) 3 instr/tick
[t= 6] ANDES-64#1: envía SUMA(10) a #2
[t= 6] ANDES-64#1: RREQ #1 buscando ruta a #2 (intento 1)
[t= 7] ORINOCO-32#2: RREQ de #1 llegó en 1 salto(s); responde RREP
[t= 8] ANDES-64#1: ruta a #2 establecida vía #2 (1 saltos)
[t= 9] ORINOCO-32#2: recibe SUMA(10) de #1 [1 salto(s), 3 ticks]
[t= 12] ANDES-64#1: envía SUMA(1000) a #3
[t= 12] ANDES-64#1: RREQ #2 buscando ruta a #3 (intento 1)
[t= 14] CARIBE-16#3: RREQ de #1 llegó en 2 salto(s); responde RREP
[t= 16] ANDES-64#1: ruta a #3 establecida vía #2 (2 saltos)
[t= 18] CARIBE-16#3: recibe SUMA(1000) de #1 [2 salto(s), 6 ticks]
[t= 34] ORINOCO-32#2: envía RESULTADO(55) a #1
```

```

[t= 35] ANDES-64#1: recibe RESULTADO(55) de #2 [1 salto(s), 1 ticks]
[t= 4363] CARIBE-16#3: envía RESULTADO(500500) a #1
[t= 4363] CARIBE-16#3: RREQ #1 buscando ruta a #1 (intento 1)
[t= 4365] ANDES-64#1: RREQ de #3 llegó en 2 salto(s); responde RREP
[t= 4367] CARIBE-16#3: ruta a #1 establecida vía #2 (2 saltos)
[t= 4369] ANDES-64#1: recibe RESULTADO(500500) de #3 [2 salto(s), 6 ticks]
[t= 4403] ANDES-64#1: la CPU ejecutó HLT

```

Métrica	Valor
ticks	4403
datos_app	4
entregados	4
latencia_media	4.0
latencia_max	6
tx_datos	6
tx_rreq	5
tx_rrep	5
descubrimientos	3
energia	ANDES-64#1: 8.0, ORINOCO-32#2: 12.0, CARIBE-16#3: 5.0
instrucciones	ANDES-64#1: 35221, ORINOCO-32#2: 22015, CARIBE-16#3: 13209

Análisis

- *Resultados correctos en las tres máquinas:* $1+\dots+10 = 55$ y $1+\dots+1000 = 500\ 500$. Este último supera el máximo de una palabra de 16 bits (65 535): el CARIBE-16 lo obtuvo sumando en doble precisión (dos palabras con propagación del acarreo mediante ADC) y lo transmitió como 4 bytes en orden de red, que el ANDES-64 reconstruyó y extendió a 64 bits. La interoperabilidad depende del formato canónico, no de la arquitectura.
- *Descubrimiento de rutas:* la ruta de 1 salto a #2 se obtuvo en 2 ticks (RREQ + RREP) y la de 2 saltos a #3 en 4 ticks. La primera solicitud tardó 3 ticks en llegar (2 de descubrimiento + 1 de transmisión) y la segunda 6. Los paquetes siguientes ya no pagan ese costo: la respuesta de #2 llegó en 1 tick.
- *Vencimiento de rutas:* el CARIBE-16 tardó 4 345 ticks (43 s simulados) en calcular la suma; para entonces su ruta de vuelta (vida de 300 ticks) había expirado y tuvo que descubrirla de nuevo (RREQ #1 del nodo #3 en $t = 4\ 363$). Es el comportamiento esperado de un protocolo reactivo.
- *Energía:* el nodo intermedio (#2) es el que más gasta (12 unidades frente a 8 y 5): en una MANET los nodos centrales pagan el reenvío de tráfico ajeno, lo que motiva protocolos

que balancean la carga.

3. Escenario 2 — Efecto de la movilidad

Seis nodos (dos de cada fabricante) con posiciones iniciales aleatorias en $300 \text{ m} \times 300 \text{ m}$ se mueven con random waypoint (pausa de 1 s). El ANDES-64 #1 envía 40 PING al CARIBE-16 #6 (uno cada $\sim 0,9 \text{ s}$) y espera cada PONG hasta $\sim 1 \text{ s}$; los demás nodos ejecutan el servidor y reenvían tráfico.

v (m/s)	PONG recibidos por semilla (1..5)	Éxito app.	PDR	Latencia media (ticks)	RREQ tx	Rupturas	Descartados
0	0/40 · 40/40 · 0/40 · 40/40 · 0/40	40,0 %	0,400	1,2	193,2	0,0	24,0
2	30/40 · 14/40 · 19/40 · 40/40 · 23/40	63,0 %	0,750	3,2	128,2	0,4	14,8
5	28/40 · 23/40 · 32/40 · 40/40 · 34/40	78,5 %	0,872	3,5	85,2	2,8	8,6
10	32/40 · 29/40 · 36/40 · 35/40 · 35/40	83,5 %	0,909	2,5	70,6	3,6	6,6
20	30/40 · 21/40 · 28/40 · 29/40 · 26/40	67,0 %	0,800	4,2	131,6	6,6	13,2

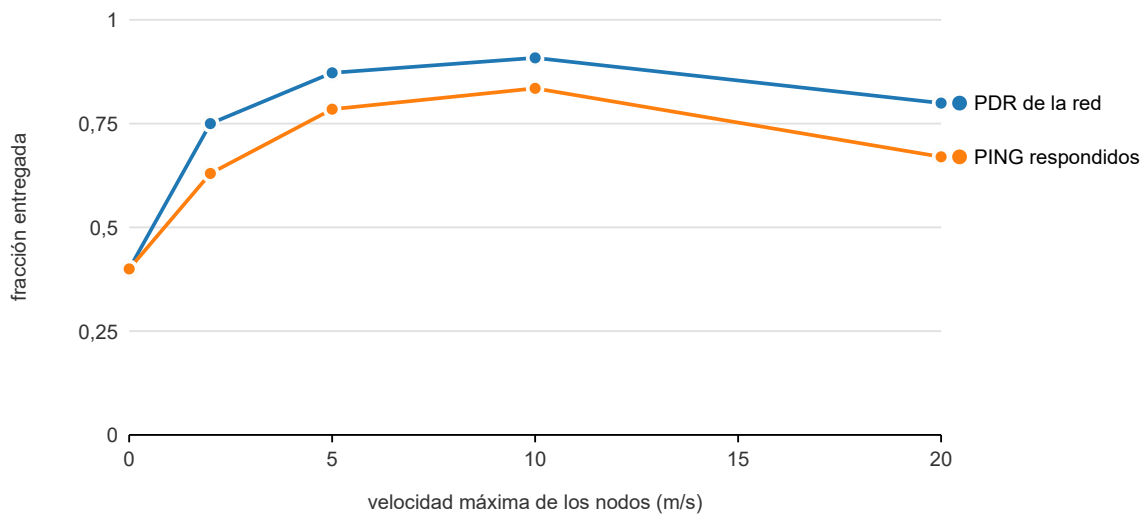


Figura 2. PDR de la red (todos los paquetes de datos) y fracción de PING respondidos a tiempo, frente a la velocidad (promedio de 5 semillas).

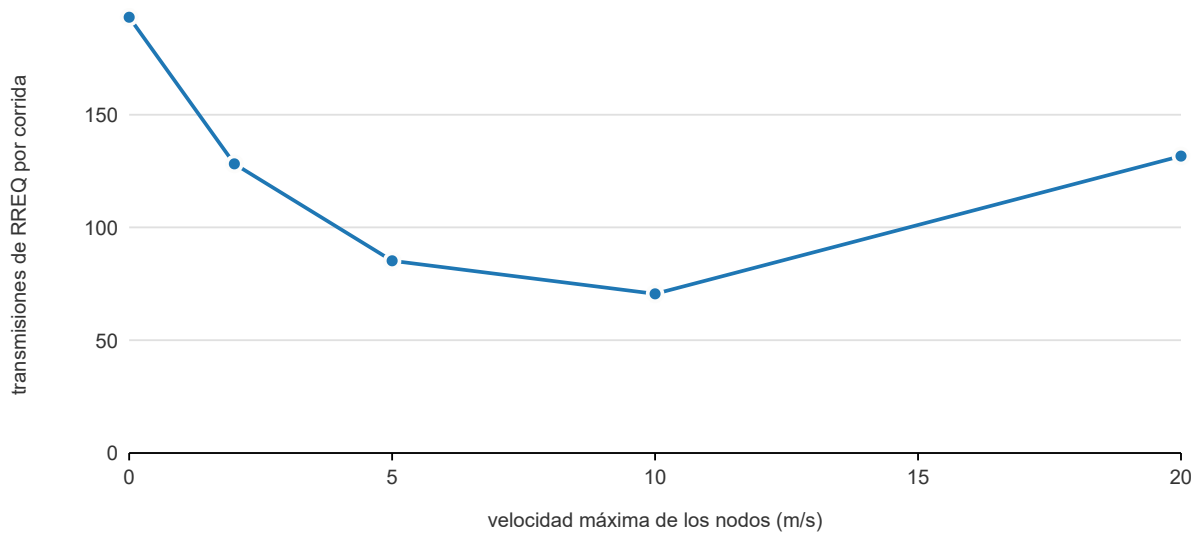


Figura 3. Transmisiones de RREQ por corrida.

Análisis

- *Red estática particionada ($v = 0$):* con 6 nodos en 9 hectáreas y 100 m de alcance, en 3 de las 5 semillas el nodo #6 quedó en una componente distinta a la del #1: 0 de 40 respuestas y 193 RREQ inútiles por corrida (cada PING dispara hasta 3 descubrimientos y termina descartado). En las otras 2 semillas la entrega fue perfecta. El promedio (40 %) oculta un comportamiento de “todo o nada”.
- *La movilidad moderada mejora la entrega:* al moverse, los nodos se reencuentran (el random waypoint además concentra los nodos en el centro del área) y las particiones son temporales: el éxito sube a 63 % (2 m/s), 78,5 % (5 m/s) y 83,5 % (10 m/s), y el control cae de 193 a 71 RREQ por corrida.
- *La movilidad alta la empeora:* a 20 m/s las rupturas de enlace casi se duplican (6,6 por corrida frente a 3,6 a 10 m/s): una ruta de 3 saltos dura poco, se pierden paquetes durante la reparación y el control vuelve a crecer (132 RREQ). Aparece así el compromiso clásico de las MANET entre conectividad y estabilidad de las rutas.
- *PDR frente a éxito de la aplicación:* el PDR cuenta PING y PONG por separado; el éxito de la aplicación exige ambos y dentro del plazo, por lo que siempre es menor (p. ej. 0,80 frente a 67 % a 20 m/s).
- La latencia media se mantuvo baja (1–4 ticks) porque la mayoría de los paquetes encuentran la ruta ya instalada; el máximo aparece en los paquetes que esperan un descubrimiento o una reparación local.

4. Escenario 3 — Enlaces con pérdidas

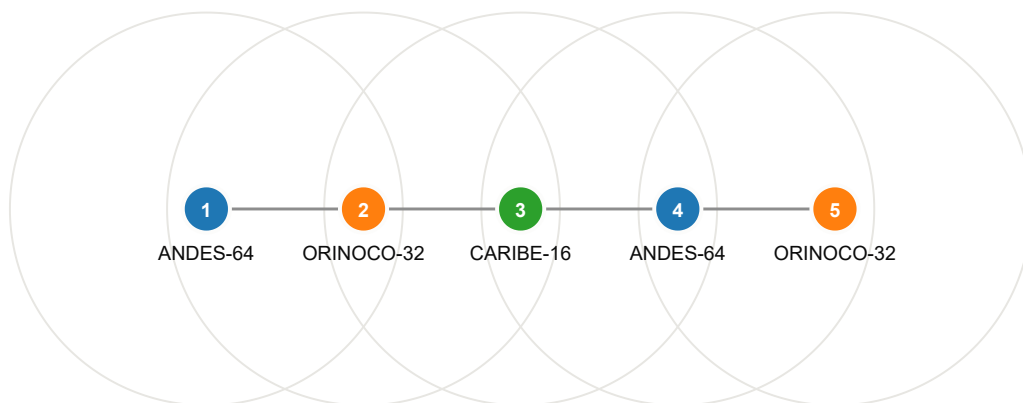


Figura 4. Cadena fija de 5 nodos separados 80 m: el camino de #1 a #5 tiene 4 saltos y atraviesa las tres arquitecturas.

El ANDES-64 #1 envía 30 PING al ORINOCO-32 #5. Cada transmisión de radio se pierde con probabilidad p .

p pérdida	PONG recibidos por semilla	Éxito app.	PDR	Latencia media	Retransmisiones	Rupturas falsas	RREQ tx
0,0	30/30 · 30/30 · 30/30 · 30/30 · 30/30	100,0 %	1,000	4,1	0,0	0,0	4,0
0,1	30/30 · 29/30 · 30/30 · 30/30 · 30/30	99,3 %	1,000	4,8	30,2	0,2	7,4
0,2	29/30 · 29/30 · 30/30 · 30/30 · 28/30	97,3 %	0,997	6,1	60,0	2,6	21,2
0,3	24/30 · 27/30 · 26/30 · 22/30 · 24/30	82,0 %	0,932	8,2	87,6	5,8	57,0
0,4	11/30 · 22/30 · 15/30 · 22/30 · 17/30	58,0 %	0,762	10,9	98,2	8,6	100,0
0,5	1/30 · 8/30 · 12/30 · 6/30 · 4/30	20,7 %	0,397	27,8	79,6	10,0	158,4

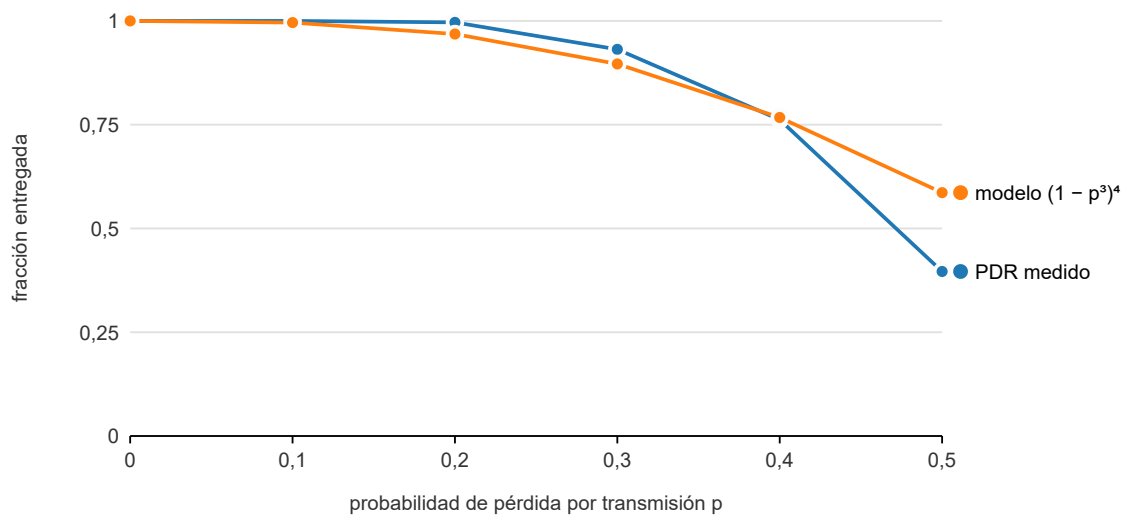


Figura 5. PDR medido frente al modelo analítico $(1 - p^3)^4$ (4 saltos con 3 intentos MAC por salto).

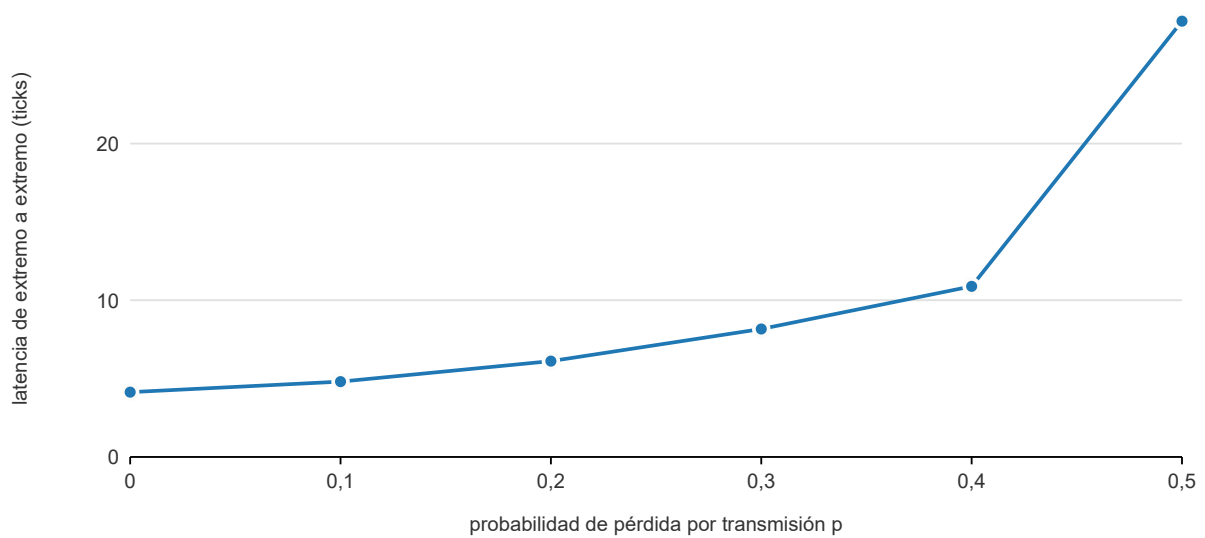


Figura 6. Latencia media de extremo a extremo.

Análisis

- *Los reintentos MAC ocultan pérdidas moderadas:* hasta $p = 0,2$ se entregó el 99,7 % de los paquetes, porque la probabilidad de que fallen 3 intentos seguidos en un salto es $p^3 = 0,008$. El costo se paga en retransmisiones (60 por corrida a $p = 0,2$) y en latencia (de 4,1 a 6,1 ticks), que crece casi linealmente con p .
- *El modelo explica la tendencia hasta $p = 0,3$:* $(1 - p^3)^4$ predice 0,897 y se midió 0,932 (la ruta a veces se aprende directamente con los datos y hay reintentos de descubrimiento que rescatan paquetes). En $p = 0,4$ medición y modelo coinciden (0,762 frente a 0,768),

pero en $p = 0,5$ el sistema cae **muy por debajo del modelo** (0,397 medido frente a 0,586 predicho).

- *Causa: rupturas falsas.* Cuando fallan los 3 intentos, la capa de enlace declara roto un enlace que sigue en alcance; AODV borra rutas válidas y lanza descubrimientos por inundación que también sufren pérdidas (158 RREQ por corrida en $p = 0,5$ frente a 4 sin pérdidas). Si 3 descubrimientos fallan, los paquetes se descartan. La pérdida en el canal se amplifica en la capa de red: es la razón por la que protocolos reales usan métricas de calidad de enlace (ETX) en lugar de conteo de saltos.
- La heterogeneidad de los nodos intermedios no afecta el reenvío: el enrutamiento lo realiza la capa de red de cada nodo, independientemente de su CPU.

5. Escenario 4 — Heterogeneidad de las CPU (tiempo de servicio)

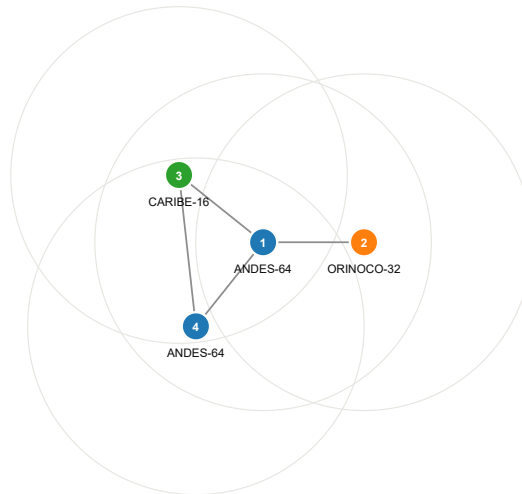


Figura 7. Estrella: el coordinador (#1, ANDES-64) a un salto de un servidor de cada fabricante.

El coordinador envía la misma SUMA(n) a los tres servidores. Tiempo de servicio = ticks entre la llegada de la solicitud al servidor y el envío de su respuesta.

n	ANDES-64 (ticks)	ORINOCO-32 (ticks)	CARIBE-16 (ticks)	Resultado	CARIBE / ANDES
10	16	25	55	55	3,4
100	72	187	445	5050	6,2
300	197	547	1311	45150	6,7
1000	635	1807	4345	500500	6,8

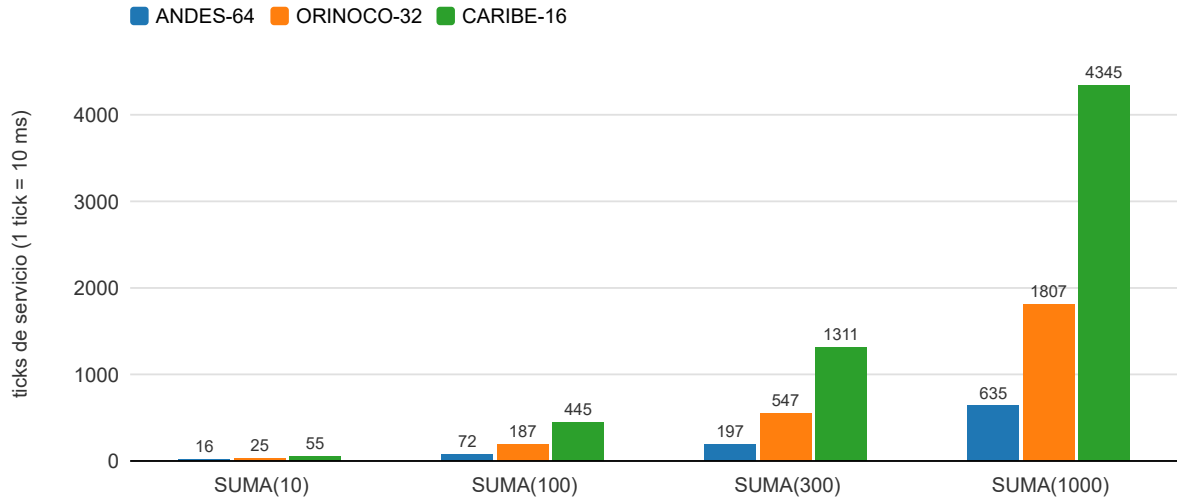


Figura 8. Tiempo de servicio de SUMA(n) por arquitectura.

Análisis

- *Mismo resultado, costos muy distintos:* las tres máquinas devolvieron exactamente el mismo valor en todos los casos (el coordinador imprimió, p. ej., 500 500 tres veces y total = 1 501 500), pero con $n = 1000$ el CARIBE-16 tardó 6,8 veces más que el ANDES-64 y el ORINOCO-32 2,8 veces más.
- *Dos causas separables:* multiplicando ticks por el reloj de cada CPU se obtienen las instrucciones ejecutadas por iteración del bucle: ANDES-64 ≈ 5 ($635 \times 8 / 1000$), ORINOCO-32 ≈ 9 ($1\,807 \times 5 / 1000$) y CARIBE-16 ≈ 13 ($4\,345 \times 3 / 1000$). Es decir, una parte de la diferencia (factor $\approx 2,6$) se debe al **conjunto de instrucciones** —la máquina de registros hace la suma y el incremento en registros; la de pila necesita DUP/ROT/SWAP para reordenar operandos; la de acumulador de 16 bits carga y guarda en memoria en cada paso y hace la suma en dos palabras— y el resto (factor $\approx 2,7$) a la **frecuencia del reloj**.
- *Tamaño de palabra:* el CARIBE-16 necesita el doble de trabajo para cada suma porque el resultado no cabe en 16 bits; si se hubiera usado aritmética de 16 bits, SUMA(1000)

habría devuelto $500\ 500 \bmod 65\ 536 = 41\ 748$ sin aviso. La heterogeneidad no solo cambia el rendimiento: si los programas no la contemplan, cambia los resultados.

- El costo fijo ($n = 10$) está dominado por la recepción y el armado de la respuesta, en el que el CARIBE-16 también es el más lento (55 ticks) por mover la PDU byte a byte a través del acumulador.

6. Conclusiones

1. El emulador permite que tres computadores von Neumann con ISA, tamaños de palabra (64/32/16 bits), orden de bytes, modelos de operandos (registros/pila/acumulador), banderas, memorias y ensambladores distintos cooperen en una MANET, ejecutando programas binarios reales de cada arquitectura.
2. La clave de la interoperabilidad es la separación en capas: la red solo transporta bytes, y un formato canónico (PDU con enteros de 32 bits en orden de red) obliga a cada programa a hacer su propia traducción (desplazamientos en ANDES-64, BSWAP en ORINOCO-32, doble precisión en CARIBE-16).
3. El enrutamiento reactivo tipo AODV encontró y reparó rutas multi-salto a través de nodos de cualquier fabricante; su desempeño depende fuertemente de la conectividad (particiones), de la velocidad (rupturas) y de la calidad de los enlaces (rupturas falsas y amplificación del control).
4. La heterogeneidad de las CPU se manifiesta como diferencias de hasta 7 veces en el tiempo de servicio de una misma operación, atribuibles tanto al repertorio de instrucciones como al reloj, y como riesgo de resultados erróneos si se ignora el tamaño de palabra.